

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN: THỰC TẬP CƠ SỞ



BÁO CÁO CUỐI KỲ

ĐỀ TÀI: XÂY DỰNG WEBSITE ĐỌC TRUYỆN

TRỰC TUYẾN METRUYEN

Sinh viên thực hiện: Nguyễn Thành Nam

Mã sinh viên: B23DCCN587

Nhóm: 11

Giảng viên hướng dẫn: Kim Ngọc Bách

Hà Nội - 2026

Mục Lục

CHƯƠNG 1: GIỚI THIỆU CHUNG	4
1. Lý do chọn đề tài	4
2. Ý nghĩa và tính ứng dụng	5
3. Giá trị học thuật	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	6
1. Cơ sở lý thuyết - MERN Stack	6
2. Công nghệ và các thư viện hỗ trợ khác	7
2.1 Cơ sở dữ liệu (Database)	7
2.2. Lập trình máy chủ (Backend)	8
2.3. Lập trình giao diện (Frontend)	9
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	12
1. Phân tích hệ thống	12
1.1. Nhóm chức năng của Guest	12
1.2. Nhóm chức năng của User	12
1.3 Nhóm chức năng của Admin	13
1.4 Biểu đồ Use Case Tổng quát	14
1.5. Kịch bản	14
2. Thiết kế hệ thống	24
2.1. Kiến trúc tổng thể	24
2.2. Mô hình dữ liệu	24
2.3. Mối quan hệ giữa các Collection	29
2.4. Danh sách API endpoints	30
2.5 Cấu trúc mã nguồn	33
CHƯƠNG 4: KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM	37
1 Môi trường cài đặt	37
2 Kết quả và kiểm thử	37
2.1. Kết quả	37
2.2. Kiểm thử	41
3. Đánh giá kết quả	48
CHƯƠNG 5: KẾT LUẬN	50
1. Kết quả đạt được	50

2. Hạn chế	50
3. Hướng phát triển	51
TÀI LIỆU THAM KHẢO	53

CHƯƠNG 1: GIỚI THIỆU CHUNG

1. Lý do chọn đề tài

Trong kỷ nguyên công nghệ số, sự dịch chuyển từ văn hóa đọc sách in truyền thống sang các nền tảng kỹ thuật số đang diễn ra mạnh mẽ. Đặc biệt, nhu cầu giải trí thông qua việc đọc truyện trực tuyến của giới trẻ ghi nhận sự tăng trưởng đột phá. Tuy nhiên, nhiều nền tảng đọc truyện hiện nay đang gặp phải các vấn đề hạn chế nhất định như: lạm dụng quảng cáo gây gián đoạn trải nghiệm, tốc độ tải trang chậm do không tối ưu hóa hình ảnh, đặc biệt là thiếu yếu tố tương tác cộng đồng. Nhận thấy những hạn chế đó, dự án "MeTruyen – Nền tảng đọc truyện online" được đề xuất và thực hiện. Mục tiêu cốt lõi của dự án là xây dựng một nền tảng đọc truyện tranh trực tuyến hiện đại, tối giản nhưng vẫn đảm bảo hiệu năng truyền tải hình ảnh và đặt trải nghiệm người dùng (UX/UI) lên hàng đầu, đồng thời cũng hỗ trợ các tính năng kết nối, tương tác cộng đồng. Đồng thời cũng là cơ hội để có thể tổng hợp và vận dụng kiến thức đã học và cóp nhặt trên trường lớp hệ thống lại và phát triển một dự án thực sự.

Mục tiêu của dự án:

- Phát triển và hoàn thiện một hệ thống web cho phép người dùng trải nghiệm việc đọc truyện tranh trực tuyến một cách mượt mà và ổn định.
- Xây dựng hệ thống quản lý dữ liệu toàn diện bao gồm: quản lý đầu truyện, danh sách chương (chapter), hệ thống bình luận, tính năng đánh dấu (bookmark) và theo dõi lịch sử đọc của từng tài khoản.
- Triển khai cơ chế phân quyền người dùng một cách chặt chẽ, chia thành các vai trò: Khách (Guest), Người dùng (User) và Quản trị viên (Admin) để dễ dàng kiểm soát.
- Tích hợp hệ thống nạp điểm thưởng (M-Point) và cơ chế nâng cấp tài khoản VIP, tạo tiền đề cho các mô hình kinh doanh trên nền tảng web thực tế.

Phạm vi của dự án:

- **Backend:** Xây dựng hệ thống RESTful API dựa trên nền tảng Node.js kết hợp với framework Express.js, đồng thời sử dụng hệ quản trị cơ sở dữ liệu MongoDB để lưu trữ thông tin.
- **Frontend:** Giao diện người dùng được thiết kế theo kiến trúc Single Page Application (SPA) thông qua thư viện React cùng build tool Vite, sử dụng Tailwind CSS để xây dựng giao diện hiện đại.

- **Bảo mật:** Áp dụng phương thức xác thực bằng JSON Web Token (JWT) và thiết lập cơ chế phân quyền truy cập theo vai trò (role-based).
- **Triển khai:** Hệ thống hiện được đóng gói và kiểm thử trên môi trường phát triển (development), sẵn sàng cho các bước cấu hình để đưa lên môi trường thực tế (production).

2. Ý nghĩa và tính ứng dụng

- Ý nghĩa thực tiễn:

- Cung cấp một nền tảng đọc truyện sạch sẽ. UI tối giản, gọn gàng, thân thiện, không quảng cáo phiền nhiễu.
- Hỗ trợ đọc truyện mọi lúc mọi nơi thông qua trình duyệt web
- Cho phép người dùng quản lý bộ sưu tập truyện cá nhân (bookmark, lịch sử)
- Hỗ trợ tương tác cộng đồng qua bình luận linh hoạt ở cấp độ truyện và chapter

- Tính ứng dụng:

- Hoàn toàn có thể triển khai thực tế như một website đọc truyện độc lập
- Kiến trúc module hóa, dễ mở rộng tính năng (chat realtime, đề xuất thông minh,...)
- Mô hình kinh doanh: nạp điểm M-Point, gói VIP cho chapter độc quyền

3. Giá trị học thuật

Dự án MeTruyen này không chỉ dừng lại ở việc tạo ra một ứng dụng thực tiễn, mà còn mang lại những giá trị học thuật sâu sắc trong lĩnh vực Công nghệ thông tin và Phát triển phần mềm. Qua quá trình tìm hiểu và thực sự bắt tay vào làm, dự án đã giúp em chuyển hóa các lý thuyết nền tảng trên giấy thành các tư duy thực hành kỹ thuật thực tế, bao gồm:

- Áp dụng thành công kiến trúc MERN Stack (bao gồm MongoDB, Express.js, React, Node.js) - một trong những bộ công nghệ phổ biến nhất để phát triển ứng dụng web hiện đại.
- Thực hành thiết kế hệ thống RESTful API theo đúng chuẩn mực (xây dựng resource-based URL, sử dụng chuẩn phương thức HTTP và status codes).
- Triển khai và áp dụng mô hình bảo mật bằng xác thực JWT (JSON Web Token) cho các API của hệ thống.
- Thực hành thiết kế cơ sở dữ liệu dạng NoSQL sử dụng MongoDB, đồng thời tối ưu hóa các câu truy vấn (query) theo những tình huống (use case) thực tế.

- Áp dụng kiến trúc dựa trên component (component-based) của React vào việc xây dựng giao diện, kết hợp quản lý state hiệu quả bằng thư viện Zustand.
- Sử dụng middleware pattern trong framework Express.js để xây dựng cơ chế xử lý lỗi tập trung (thông qua việc sử dụng asyncHandler và AppError).
- Tìm hiểu và tích hợp trực tiếp các dịch vụ của bên thứ ba (third-party), điển hình là dịch vụ Cloudinary để lưu trữ và quản lý tài nguyên hình ảnh.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

1. Cơ sở lý thuyết - MERN Stack

MERN Stack là một tập hợp các công nghệ phát triển web toàn diện (full-stack) bao gồm bốn công nghệ cốt lõi: **MongoDB**, **Express.js**, **React** và **Node.js**. Sự kết hợp của bốn công nghệ này tạo nên một trong những stack công nghệ được ưa chuộng và sử dụng phổ biến nhất hiện nay trong lĩnh vực phát triển phần mềm. [1]

Ưu điểm lớn nhất của MERN Stack nằm ở sự đồng nhất về mặt ngôn ngữ lập trình. Việc sử dụng JavaScript (hoặc TypeScript) xuyên suốt từ phía máy khách (client-side) cho đến máy chủ (server-side) giúp quá trình phát triển trở nên liền mạch, tối ưu hóa thời gian xây dựng và dễ dàng bảo trì hệ thống. [1]

- Trong dự án này, các khái niệm lý thuyết cốt lõi được áp dụng bao gồm:

- **Kiến trúc Client-Server:** Hệ thống web hiện đại trong dự án được thiết kế vận hành dựa trên mô hình Client-Server[2]. Cụ thể mô hình này hoạt động với các thành phần:
 - **Client (Frontend):** Đóng vai trò là phía trình duyệt của người dùng, chịu trách nhiệm thực thi ứng dụng React và gửi các yêu cầu giao tiếp với máy chủ thông qua giao thức HTTP.
 - **Server (Backend):** Là máy chủ vận hành trên nền tảng Node.js kết hợp framework Express.js, đảm nhận việc tiếp nhận, xử lý các yêu cầu (request) từ phía Client và phản hồi kết quả về dưới định dạng cấu trúc dữ liệu JSON.
 - **Database:** Hệ quản trị cơ sở dữ liệu MongoDB đảm nhận việc lưu trữ dữ liệu của toàn bộ hệ thống. Phía Server sẽ tiến hành kết nối và tương tác với cơ sở dữ liệu này thông qua công cụ Mongoose ODM.
- **Kiến trúc RESTful API:** Hệ thống ứng dụng chuẩn kiến trúc REST (Representational State Transfer) để thiết kế giao diện lập trình ứng dụng (API). Kiến trúc này xoay quanh khái niệm quản lý dựa trên tài nguyên

(resource) và ứng dụng nghiêm ngặt các phương thức giao tiếp chuẩn của giao thức HTTP (GET, POST, PUT, và DELETE). Theo nguyên tắc định tuyến của chuẩn REST, mỗi điểm cuối (endpoint) sẽ đại diện trực tiếp cho một tài nguyên xác định trong hệ thống và được định danh một cách độc lập bằng một đường dẫn URL duy nhất [3].

- **Mô hình ứng dụng trang đơn (Single Page Application - SPA):** Đây là phương pháp thiết kế ứng dụng web hiện đại, trong đó hệ thống chỉ tiến hành tải một tài liệu HTML duy nhất ở lần truy cập ban đầu. Kể từ đó, mọi tác vụ phân luồng điều hướng, cập nhật và hiển thị nội dung động đều được JavaScript xử lý ngầm mà không yêu cầu trình duyệt phải tải lại (reload) toàn bộ trang web. Để đạt được điều này, thư viện React sử dụng cơ chế Virtual DOM (DOM ảo) nhằm tối ưu hóa hiệu năng kết xuất (render), chỉ cập nhật đúng các thành phần UI có sự thay đổi [4].
- **Cơ sở dữ liệu phi quan hệ (NoSQL Database):** Dự án sử dụng MongoDB - một hệ quản trị cơ sở dữ liệu NoSQL được thiết kế theo hướng tài liệu (Document-oriented). Dữ liệu trong hệ thống này được lưu trữ linh hoạt dưới dạng cấu trúc BSON (Binary JSON). Điểm khác biệt cốt lõi so với các hệ quản trị cơ sở dữ liệu SQL truyền thống là MongoDB không tổ chức dữ liệu theo dạng bảng hay ràng buộc bằng khóa ngoại. Thay vào đó, nó sử dụng các tập hợp (collection) độc lập và ứng dụng kỹ thuật tham chiếu (reference) thông qua cơ chế thẻ định danh ObjectId để tạo ra sự liên kết logic giữa các luồng dữ liệu [5].

2. Công nghệ và các thư viện hỗ trợ khác

Để hiện thực hóa các yêu cầu của hệ thống, dự án đã lựa chọn và tích hợp một loạt các công nghệ, thư viện hiện đại và tối ưu nhất cho từng thành phần.

2.1 Cơ sở dữ liệu (Database)

Hệ thống lưu trữ dữ liệu đóng vai trò nền tảng, đòi hỏi tính linh hoạt cao để đáp ứng sự thay đổi liên tục của các luồng dữ liệu (chẳng hạn như thông tin truyện, chương, và bình luận). Do đó, dự án đã sử dụng:

- **MongoDB:** Đây là hệ quản trị cơ sở dữ liệu NoSQL định hướng tài liệu (document-oriented) hàng đầu hiện nay. Thay vì sử dụng bảng và cột như SQL, MongoDB lưu trữ dữ liệu dưới định dạng tương tự JSON, mang lại sự linh hoạt tối đa trong việc thiết kế và thay đổi cấu trúc schema mà không làm gián

đoạn hệ thống. MongoDB hỗ trợ mạnh mẽ các kỹ thuật nâng cao như lập chỉ mục (indexing) để tăng tốc độ truy vấn, aggregation pipeline để xử lý dữ liệu phức tạp, và cơ chế sao chép (replication) đảm bảo tính sẵn sàng cao [5].

Đặc biệt, dự án kết hợp sử dụng nền tảng đám mây **MongoDB Atlas** (gói miễn phí 512MB) cho môi trường triển khai thực tế, việc chuyển đổi giữa cơ sở dữ liệu cục bộ (local) và đám mây (cloud) được thực hiện liền mạch chỉ thông qua việc thay đổi chuỗi kết nối MONGODB_URI trong tệp cấu hình .env.

- **Mongoose:** Để giao tiếp hiệu quả với MongoDB từ môi trường Node.js, dự án sử dụng Mongoose - một thư viện ODM (Object Data Modeling). Mongoose cung cấp một lớp định nghĩa cấu trúc (schema validation) chặt chẽ ngay tại tầng ứng dụng, giúp kiểm soát tính toàn vẹn của dữ liệu trước khi lưu vào cơ sở dữ liệu. Thêm vào đó, thư viện này cung cấp các cơ chế middleware (pre/post hooks) để tự động hóa các tác vụ và tính năng populate cho phép thực hiện các phép kết nối (JOIN ảo) giữa các bộ tài liệu khác nhau (ví dụ: lấy thông tin User từ ID trong bảng Comment) một cách tiện lợi và tối ưu [6].

2.2. Lập trình máy chủ (Backend)

Tầng máy chủ chịu trách nhiệm xử lý logic nghiệp vụ, xác thực và giao tiếp với cơ sở dữ liệu, được xây dựng dựa trên các công nghệ sau:

- **Node.js (Phiên bản 20.x):** Là một môi trường thực thi (runtime) mã JavaScript phía máy chủ, được xây dựng trên lõi V8 engine của Google Chrome. Với đặc thù sử dụng mô hình I/O không chặn (non-blocking I/O) và kiến trúc hướng sự kiện (event-driven), Node.js cực kỳ tối ưu cho các ứng dụng yêu cầu xử lý dữ liệu theo thời gian thực (realtime) và các hệ thống RESTful API phục vụ lượng lớn request đồng thời [7].
- **Express.js (Phiên bản 4.x):** Là một web framework tối giản nhưng mạnh mẽ dành cho Node.js. Express.js cung cấp bộ công cụ toàn diện để thiết lập hệ thống định tuyến (routing), quản lý các middleware, và xây dựng cơ chế xử lý lỗi (error handling) một cách có hệ thống, giúp mã nguồn trở nên gọn gàng và dễ bảo trì [7].
- **Cơ chế bảo mật và Phân quyền:**
 - **JSON Web Token (JWT):** Áp dụng chuẩn mở (RFC 7519) để xây dựng cơ chế xác thực người dùng không trạng thái. Sau khi đăng nhập thành

công, server sẽ sinh ra một token chứa thông tin định danh của người dùng (payload) được mã hóa bằng một khóa bí mật (secret key) kèm theo thời gian hết hạn. Thư viện jsonwebtoken được sử dụng để tạo và giải mã các token này trong các request tiếp theo để kiểm tra quyền truy cập (Authorization) [8].

- **Quản lý tài nguyên đa phương tiện:**

- **Cloudinary:** Dịch vụ lưu trữ đám mây chuyên biệt để quản lý, tối ưu hóa và phân phối tài nguyên hình ảnh (ảnh bìa truyện, ảnh chương truyện, avatar người dùng). Cloudinary cung cấp bộ Node.js SDK cho phép hệ thống gọi API để tải lên, xóa hoặc áp dụng các bộ lọc chuyển đổi hình ảnh tự động [9].
- **Multer:** Kết hợp với Cloudinary, dự án sử dụng Multer đóng vai trò là middleware xử lý dữ liệu dạng multipart/form-data. Multer được cấu hình sử dụng bộ nhớ đệm (memory storage) để tiếp nhận tệp tin từ client và đẩy trực tiếp lên Cloudinary thông qua multer-storage-cloudinary, bỏ qua bước lưu trữ tạm thời trên ổ cứng máy chủ [10].

- **Các thư viện tiện ích bổ trợ:**

- **archiver & axios:** Thư viện archiver hỗ trợ nén tệp tin thành định dạng ZIP trực tiếp trên luồng bộ nhớ (stream). Kết hợp cùng axios (HTTP client), hệ thống có khả năng tải xuống (fetch) hàng loạt ảnh từ Cloudinary, nén thành tệp ZIP và trả về cho người dùng khi họ có nhu cầu tải nguyên một chương truyện về máy [11].
- **slugify:** Công cụ chuyển đổi chuỗi văn bản thông thường thành các đoạn mã định danh thân thiện với URL (slug). Việc tạo slug cho tên truyện và thể loại không chỉ giúp URL đẹp mắt, dễ đọc mà còn hỗ trợ tối ưu hóa công cụ tìm kiếm (SEO) [12].
- **cors:** Middleware bắt buộc để xử lý các chính sách Chia sẻ tài nguyên chéo nguồn (Cross-Origin Resource Sharing). Nó cho phép ứng dụng Frontend (chạy ở cổng 3000) được phép gửi request hợp lệ đến máy chủ Backend (chạy ở cổng 5000) mà không bị trình duyệt ngăn chặn [13].

2.3. Lập trình giao diện (Frontend)

Tầng giao diện người dùng (Client-side) đóng vai trò quyết định đến trải nghiệm trực tiếp của độc giả. Để đáp ứng tiêu chí mượt mà và tối ưu hóa hiệu năng, dự án đã áp dụng một loạt các thư viện và công cụ Frontend hiện đại:

- **React (Phiên bản 18.x):** Là một thư viện JavaScript mạnh mẽ chuyên biệt cho việc xây dựng giao diện người dùng. React hoạt động dựa trên cơ chế kết xuất khai báo (declarative) và kiến trúc hướng thành phần (component-based), cho phép chia nhỏ các khu vực trên giao diện web thành các module độc lập, dễ bảo trì và có tính tái sử dụng cao. Thông qua thuật toán Virtual DOM, React tối ưu hóa quá trình cập nhật giao diện bằng cách chỉ render lại những phần tử DOM có sự thay đổi thực sự thay vì tải lại toàn bộ trang [14].
- **Vite:** Dự án sử dụng Vite làm công cụ đóng gói (build tool) và máy chủ phát triển (dev server) thế hệ mới. Tích hợp sẵn React plugin, Vite cung cấp tốc độ khởi động dự án cực nhanh và hỗ trợ cơ chế thay thế module nóng (Hot Module Replacement - HMR) tức thời, giúp lập trình viên xem ngay lập tức các thay đổi mã nguồn. Đồng thời, công cụ này cũng được cấu hình proxy để điều hướng các API request trong môi trường phát triển (development), giải quyết triệt để vấn đề CORS khi Frontend và Backend chạy ở hai cổng (port) khác nhau [15].
- **Tailwind CSS:** Để thống nhất phong cách thiết kế, dự án sử dụng framework CSS theo hướng tiện ích (utility-first) này. Việc xây dựng giao diện bằng các class có sẵn giúp tiết kiệm thời gian viết mã CSS tùy chỉnh, giữ cho kích thước tệp CSS ở mức tối thiểu. Hệ thống đã ứng dụng Tailwind để cấu hình một bộ giao diện nền tối (dark theme) nhất quán và tùy biến các font chữ đặc thù nhằm bảo vệ mắt người đọc [16].
- **Zustand:** Đối với bài toán quản lý trạng thái toàn cục (state management), đặc biệt là trạng thái xác thực (auth store), dự án lựa chọn Zustand nhờ dung lượng siêu nhẹ và API đơn giản dựa trên hooks. Công cụ này giúp lưu trữ thông tin User và Token một cách an toàn, cho phép các component truy xuất dữ liệu người dùng ở bất kỳ đâu trong ứng dụng [17].
- **React Router v6:** Đây là thư viện định tuyến (routing) chuẩn mực để điều hướng giữa các trang trong kiến trúc SPA. Phiên bản 6 cung cấp các tính năng quản lý route lồng nhau (nested routes) và layout routes hiệu quả. Quan trọng hơn, thư viện này được dùng để thiết lập các tuyến đường bảo mật (protected routes), qua đó ngăn chặn các truy cập trái phép vào trang đọc truyện (yêu cầu User) hoặc phân hệ quản trị (yêu cầu Admin) [18].
- **Axios (Client-side):** Đóng vai trò là HTTP client trên trình duyệt, Axios được cấu hình với các interceptor để tự động đính kèm Bearer token vào phần Header của mọi request gửi lên server. Chức năng này cũng giúp tự động điều

hướng người dùng quay lại trang đăng nhập nếu máy chủ trả về lỗi 401 (chưa xác thực hoặc token hết hạn) [11].

- **Các thư viện UI/UX hỗ trợ:**

- **react-hot-toast:** Thư viện tạo thông báo (toast notification) dạng pop-up gọn nhẹ, dùng để phản hồi trực quan các trạng thái thành công hoặc báo lỗi cho người dùng [19].
- **react-icons:** Cung cấp bộ sưu tập icon phong phú từ các thư viện phổ biến (như Font Awesome, Material Design) dưới dạng các component React, giúp giao diện trở nên sinh động hơn [20].
- **react-dropzone:** Hỗ trợ tính năng kéo thả (drag & drop) tập tin một cách trực quan. Thư viện này được ứng dụng vào trang quản trị (Admin), cho phép kéo thả nguyên một thư mục chứa hình ảnh để tự động tải lên (upload) toàn bộ các trang của một chương truyện [21].

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

1. Phân tích hệ thống

Hệ thống MeTruyen được thiết kế để phục vụ ba nhóm người dùng chính tương ứng với ba mức độ phân quyền khác nhau:

- **Guest** (khách chưa đăng nhập)
- **User** (người dùng đã đăng nhập)
- **Admin** (quản trị viên)

Việc phân chia này giúp hệ thống quản lý luồng truy cập chặt chẽ và cung cấp trải nghiệm cá nhân hóa tốt nhất.

1.1. Nhóm chức năng của Guest

Khách truy cập vẫn lại là những người dùng chưa có tài khoản hoặc chưa tiến hành đăng nhập vào hệ thống. Nhóm người dùng này được phép thực hiện các thao tác cơ bản để tìm hiểu và khám phá nội dung trên nền tảng.

- Các chức năng cụ thể bao gồm:

- **Xem danh sách truyện:** Duyệt truyện theo các thể loại khác nhau, tìm kiếm và sắp xếp danh sách.
- **Xem chi tiết truyện:** Xem các thông tin tổng quan của truyện và danh sách các chapter hiện có.
- **Xem bình luận:** Đọc các bình luận do người khác đăng tải (không được phép đăng bình luận).
- **Đăng ký/Đăng nhập:** Tạo tài khoản mới hoặc đăng nhập vào hệ thống.

1.2. Nhóm chức năng của User

Đây là nhóm người dùng đã hoàn tất việc đăng nhập, được cấp quyền khai thác tối đa các tiện ích đọc truyện và tương tác cộng đồng của nền tảng. Nhóm này kế thừa toàn bộ các quyền của Guest và được bổ sung thêm các tính năng sau:

- **Đọc chapter:** Xem nội dung chi tiết của các chapter
- **Tương tác cộng đồng (Bình luận):** Đăng tải hoặc xóa bình luận của chính mình ở cấp độ truyện và cấp độ chapter.
- **Quản lý thư viện cá nhân:**
 - **Bookmark:** Lưu các bộ truyện yêu thích để quản lý bộ sưu tập.
 - **Lịch sử đọc:** Theo dõi và lưu lại chapter đã đọc gần nhất.
- **Quản lý tài khoản:**

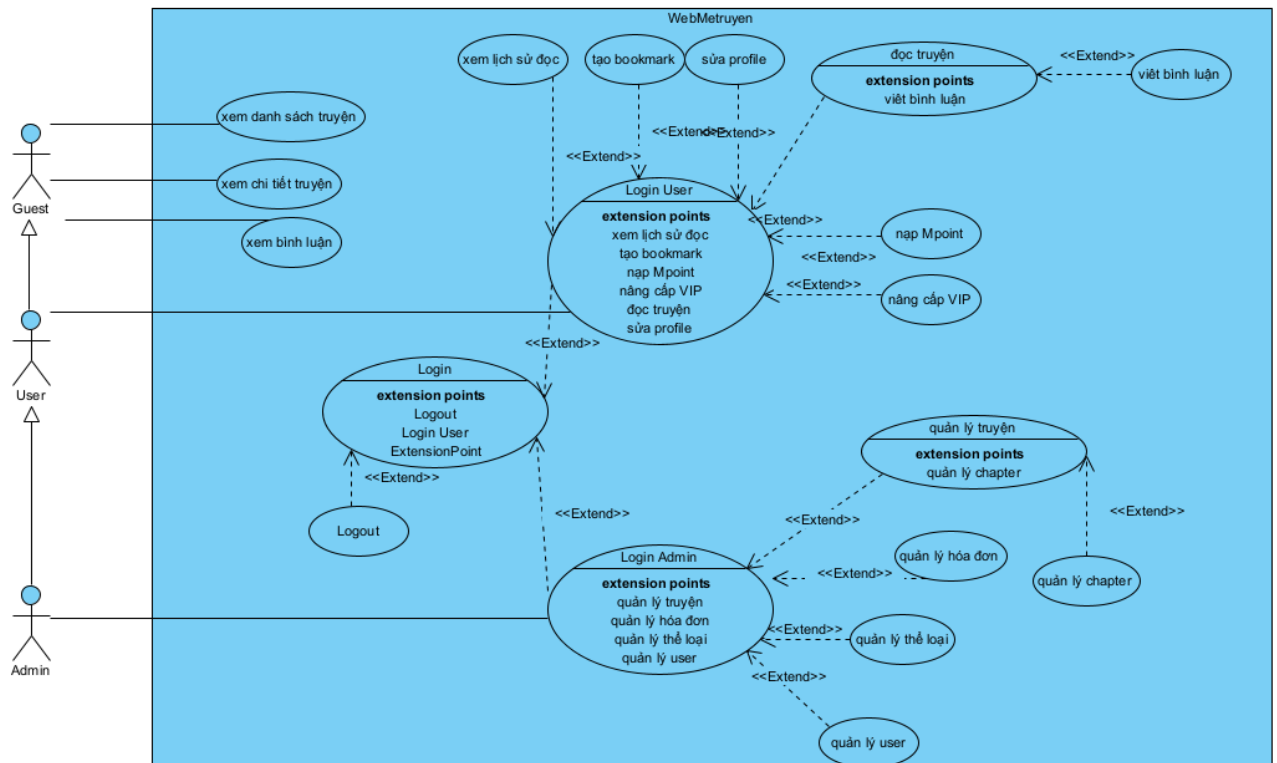
- **Chỉnh sửa profile:** Đổi tên hiển thị, mô tả (bio) và ảnh đại diện (avatar).
- **Đổi mật khẩu:** Thay đổi mật khẩu đăng nhập.
- **Hệ thống nâng cao:**
 - **Nạp M-Point:** Nạp điểm để sử dụng các dịch vụ trên hệ thống.
 - **Nâng cấp VIP:** Mua các gói VIP để đọc chapter độc quyền.

1.3 Nhóm chức năng của Admin

Quản trị viên là tài khoản có đặc quyền cao nhất, chịu trách nhiệm vận hành, giám sát và quản lý toàn bộ nội dung cũng như người dùng trên hệ thống. Admin kế thừa mọi quyền của User và sở hữu thêm bộ công cụ quản lý chuyên sâu:

- **Dashboard:** Xem thống kê tổng quan của hệ thống (số lượng user, truyện, lượt xem).
- **Quản lý nội dung:**
 - **Quản lý truyện:** Thực hiện các thao tác CRUD (tạo, đọc, sửa, xóa) đối với truyện; hỗ trợ upload chapter bằng thao tác kéo thả (drag-drop) folder.
 - **Quản lý chapter:** Sửa, xóa chapter; thực hiện đánh dấu các chapter VIP.
 - **Quản lý thể loại:** Thực hiện các thao tác CRUD đối với danh mục thể loại.
- **Quản lý người dùng và tương tác:**
 - **Quản lý user:** Xem danh sách, sửa thông tin, xóa user hoặc đặt lại mật khẩu.
 - **Quản lý bình luận:** Xóa các bình luận của người dùng trên toàn hệ thống.
- **Quản lý thanh toán:** Duyệt các giao dịch nạp điểm từ phía người dùng.

1.4 Biểu đồ Use Case Tổng quát



Hình 3.1: Biểu đồ UC tổng quát toàn hệ thống

1.5. Kịch bản

UC01: Đăng ký tài khoản

Tên	Đăng ký tài khoản
Tác nhân chính	Guest (khách chưa đăng nhập)
Mô tả	Cho phép người dùng mới tạo tài khoản để sử dụng các chức năng yêu cầu xác thực
Tiền điều kiện	Người dùng chưa đăng nhập. Username và email chưa tồn tại trong hệ thống
Hậu điều kiện	Tài khoản mới được tạo trong CSDL. Người dùng được chuyển hướng đến trang đăng nhập

- Kịch bản chính

1. Người dùng truy cập trang đăng ký (`/register`)
2. Hệ thống hiển thị form gồm: username, email, password, confirm password
3. Người dùng nhập thông tin và nhấn "Đăng ký"
4. Frontend kiểm tra confirm password khớp với password

- Frontend gửi request `POST /api/auth/register` với `{ username, email, password }`
- Backend kiểm tra username và email có bị trùng không
- Backend tạo user mới trong collection `users`
- Backend tạo JWT token và trả về `{ success: true, data: { user, token } }`
- Frontend hiển thị thông báo "Đăng ký thành công"
- Sau 1 giây, chuyển hướng đến trang đăng nhập (`/login`)

- Ngoại lệ

Tình huống	Xử lý
Username đã tồn tại	Backend trả về 400: "Email hoặc username đã được sử dụng". Hiển thị lỗi trên form
Email đã tồn tại	Backend trả về 400: "Email hoặc username đã được sử dụng". Hiển thị lỗi trên form
Password < 6 ký tự	Mongoose validation error → Backend trả về 400. Hiển thị lỗi
Confirm password không khớp	Frontend kiểm tra, hiển thị "Mật khẩu xác nhận không khớp", không gửi request
Email không đúng định dạng	Mongoose validation error (regex match). Hiển thị lỗi

UC02: Đăng nhập

Tên	Đăng nhập
Tác nhân chính	Guest
Mô tả	Cho phép người dùng đăng nhập bằng email và mật khẩu để nhận JWT token
Tiền điều kiện	Tài khoản đã tồn tại. Người dùng chưa đăng nhập
Hậu điều kiện	JWT token được lưu vào localStorage. Người dùng có quyền truy cập các route yêu cầu xác thực

- Kịch bản chính

- Người dùng truy cập trang đăng nhập (`/login`)
- Hệ thống hiển thị form gồm: email, password
- Người dùng nhập email và password, nhấn "Đăng nhập"
- Frontend gửi request `POST /api/auth/login` với `{ email, password }`
- Backend tìm user trong collection `users` với email tương ứng
(`.select('+password')`)
- Backend so sánh password plaintext (do bcryptjs đã xóa)

7. Nếu hợp lệ, backend tạo JWT token (payload: `{ id: user.\_id }`, expires: 7 ngày)
8. Backend trả về `{ success: true, data: { user, token } }`
9. Frontend lưu token vào localStorage với key `metruyen\_token`
10. Frontend lưu thông tin user vào Zustand store
11. Chuyển hướng về trang chủ (`/`)

- Ngoại lệ

Tình huống	Xử lý
Email không tồn tại	Backend trả về 401: "Email hoặc mật khẩu không đúng". Hiển thị lỗi trên form
Password sai	Backend trả về 401: "Email hoặc mật khẩu không đúng". Hiển thị lỗi trên form
Token hết hạn	Interceptor Axios bắt 401, xóa token khỏi localStorage, chuyển hướng về `/login`

UC03: Xem danh sách truyện

Tên	Xem danh sách truyện
Tác nhân chính	Guest, User
Mô tả	Duyệt danh sách truyện với bộ lọc thể loại, tìm kiếm, sắp xếp, phân trang
Tiền điều kiện	Không
Hậu điều kiện	Danh sách truyện được hiển thị trên giao diện

Kịch bản chính

1. Người dùng truy cập trang `/comics`
2. Frontend gửi request `GET /api/comics` (mặc định page=1, sort mới nhất)
3. Backend query collection `comics` với các bộ lọc, populate categories, sort, skip/limit
4. Backend trả về danh sách truyện kèm thông tin phân trang
5. Frontend render grid card, mỗi card hiển thị: ảnh bìa, tên, tác giả, views, bookmarks
6. Người dùng có thể:
 - Chọn thể loại (pill buttons) → gửi lại request với `category` query
 - Nhập từ khóa tìm kiếm → gửi request với `search` query
 - Chọn sắp xếp (mới nhất, xem nhiều, đánh dấu nhiều)
 - Chuyển trang qua phân trang

Ngoại lệ

Tình huống	Xử lý
Không có truyện nào	Hiển thị thông báo "Không tìm thấy truyện"
Trang vượt quá tổng số trang	Backend trả về trang cuối cùng
API lỗi	Hiển thị toast lỗi, giữ nguyên danh sách hiện tại

UC04: Đọc chapter

Tác nhân chính	User
Mô tả	Xem nội dung chi tiết của một chapter (danh sách ảnh pages)
Tiền điều kiện	Người dùng đã đăng nhập. Chapter tồn tại
Hậu điều kiện	Lượt xem chapter tăng 1. Lịch sử đọc được ghi lại (server). Nếu chapter yêu cầu VIP, kiểm tra quyền trước khi hiển thị

Kịch bản chính

1. Người dùng click vào một chapter từ trang chi tiết truyện hoặc từ link trực tiếp
2. Frontend gửi request `GET /api/chapters/:id` (kèm Bearer token)
3. Backend tìm chapter trong collection `chapters`, populate `comicId`, `previousChapter`, `nextChapter`
4. Backend tăng `chapter.views` lên 1
5. Backend trả về dữ liệu chapter gồm danh sách pages (url ảnh)
6. Frontend render ảnh dạng vertical scrolling
7. Frontend gửi request `POST /api/users/library/history/:chapterId` (với body `{}`)
8. Backend xóa lịch sử cũ của comic này, thêm entry mới (vị trí đầu, giới hạn 100)
9. Người dùng có thể điều hướng chapter trước/sau qua link

Kiểm tra VIP (luồng rẽ nhánh)

1. Backend kiểm tra `chapter.isVIPOnly === true`
2. Nếu true, kiểm tra `req.user.isVIP && req.user.vipExpireDate > now`
3. Nếu không có VIP → trả về 403: "Chapter này yêu cầu tài khoản VIP"
4. Frontend hiển thị thông báo lỗi, không hiển thị ảnh

Ngoại lệ

Tình huống	Xử lý
Chapter không tồn tại	Backend trả về 404. Hiển thị "Không thể tải chương"
Chapter yêu cầu VIP, user không có VIP	Backend trả về 403. Hiển thị thông báo VIP
Token hết hạn	Interceptor Axios chuyển hướng về `/login`
ChapterId là "undefined"	Frontend bỏ qua, không gọi API

UC05: Bình luận

Tác nhân chính	User
Mô tả	Đăng bình luận ở cấp truyện hoặc chapter, xóa bình luận của chính mình
Tiền điều kiện	Người dùng đã đăng nhập
Hậu điều kiện	Bình luận mới được lưu vào collection `comments`

Kịch bản chính (đăng bình luận)

1. Người dùng scroll đến phần bình luận ở trang chi tiết truyện hoặc cuối trang đọc
2. Nhập nội dung vào textarea (tối đa 1000 ký tự)
3. Nhấn "Gửi"
4. Frontend gửi request `POST /api/comics/:comicId/comments` với `{ content, chapterId? }`
5. Backend tạo comment mới trong collection `comments`
6. Backend trả về comment vừa tạo
7. Frontend thêm comment vào đầu danh sách (hoặc reload)

Kịch bản chính (xóa bình luận)

1. Người dùng hover vào bình luận của mình
2. Click nút "Xóa"
3. Frontend gửi request `DELETE /api/comics/:comicId/comments/:commentId`
4. Backend kiểm tra `comment.userId === req.user.\_id`
5. Backend xóa comment
6. Frontend xóa comment khỏi danh sách

Ngoại lệ

Tình huống	Xử lý
Nội dung rỗng	Frontend không cho gửi
Nội dung > 1000 ký tự	Mongoose validation error → Backend trả về 400
Xóa bình luận của người khác	Backend trả về 403
ComicId không hợp lệ	Backend trả về 404

UC06: Bookmark truyện

Mục	Nội dung
ID	UC06
Tên	Bookmark truyện
Tác nhân chính	User
Mô tả	Thêm hoặc xóa truyện khỏi danh sách yêu thích (toggle)
Tiền điều kiện	Người dùng đã đăng nhập. Truyện tồn tại
Hậu điều kiện	Trạng thái bookmark thay đổi. `statistics.totalBookmarks` của truyện được cập nhật (+1/-1)

Kịch bản chính

1. Người dùng vào trang chi tiết truyện
2. Click nút "Yêu thích" (hình trái tim)
3. Frontend gửi request `POST /api/users/library/bookmark/:comicId`
4. Backend tìm user, kiểm tra bookmark đã tồn tại chưa
5. Nếu chưa: thêm `{ comicId, addedAt }` vào đầu mảng `library.bookmarks`, tăng `totalBookmarks`
6. Nếu đã có: xóa khỏi mảng, giảm `totalBookmarks`
7. Backend trả về mảng bookmarks mới
8. Frontend cập nhật UI (đổi màu tim, cập nhật số lượng)

Ngoại lệ

Tình huống	Xử lý
ComicId không hợp lệ	Backend trả về 404
ComicId không tồn tại	Backend trả về 404

UC07: Nạp M-Point

Mục	Nội dung
ID	UC07
Tên	Nạp M-Point
Tác nhân chính	User
Mô tả	Nạp điểm vào tài khoản thông qua mã giao dịch
Tiền điều kiện	Người dùng đã đăng nhập
Hậu điều kiện	Số dư `mPoints` tăng lên. Giao dịch mới được tạo trong collection `payments`

Kịch bản chính

1. Người dùng vào trang nạp point (`/recharge`)
2. Nhập số tiền, phương thức thanh toán, mã giao dịch
3. Nhấn "Nạp"
4. Frontend gửi request `POST /api/users/recharge-mpoints` với `{ amount, paymentMethod, transactionId }`
5. Backend kiểm tra `transactionId` không trùng
6. Backend tạo payment mới (status: 'success')
7. Backend cộng `amount` vào `user.mPoints`
8. Trả về số dư mới

Ngoại lệ

Tình huống	Xử lý
Số tiền ≤ 0	Backend trả về 400
Mã giao dịch trống	Backend trả về 400
Mã giao dịch đã tồn tại	Backend trả về 400: "Mã giao dịch đã được sử dụng"

UC08: Nâng cấp VIP

Mục	Nội dung
ID	UC08
Tên	Nâng cấp VIP
Tác nhân chính	User
Mô tả	Mua gói VIP để đọc chapter độc quyền. Có 5 gói: 1 ngày (10k), 7 ngày (50k), 30 ngày (150k), 180 ngày (700k), 365 ngày (1tr)
Tiền điều kiện	Đã đăng nhập. Đủ M-Point
Hậu điều kiện	`user.isVIP = true`. `user.vipExpireDate` được gia hạn. `user.mPoints` giảm

Kịch bản chính

1. Người dùng chọn gói VIP
2. Frontend gửi request `POST /api/users/upgrade-vip` với `{ duration, points }`
3. Backend kiểm tra gói hợp lệ (khớp với bảng giá cố định)
4. Backend kiểm tra `user.mPoints >= points`
5. Backend trừ điểm, cập nhật `isVIP`, tính toán `vipExpireDate` (cộng dồn nếu VIP còn hạn)
6. Trả về hạn VIP mới

Ngoại lệ

Tình huống	Xử lý
Gói VIP không hợp lệ	Backend trả về 400
Không đủ M-Point	Backend trả về 400: "Không đủ M-Point"

UC09: Quản lý truyện (Admin)

Mục	Nội dung
ID	UC09
Tên	Quản lý truyện
Tác nhân chính	Admin
Mô tả	Thêm, sửa, xóa truyện. Upload chapter với drag-drop ảnh

Tiền điều kiện	Admin đã đăng nhập
Hậu điều kiện	Dữ liệu truyện/chapter thay đổi trong CSDL. Ảnh upload lên Cloudinary

Kịch bản chính (tạo truyện)

1. Admin vào trang tạo truyện (`/admin/comics/new`)
2. Nhập thông tin: tên, tác giả, mô tả, thể loại, trạng thái, năm XB
3. Upload ảnh bìa (file ảnh $\leq 5\text{MB}$)
4. Nhấn "Tạo truyện"
5. Frontend gửi FormData `POST /api/comics` (multipart, kèm coverImage)
6. Backend upload ảnh lên Cloudinary, lưu `{ url, publicId }`
7. Backend tạo comic mới, tăng `comicCount` của category
8. Chuyển hướng đến danh sách truyện

Kịch bản chính (upload chapter)

1. Admin vào trang upload chapter (`/admin/comics/:comicId/chapters/new`)
2. Nhập số chapter, tên (tùy chọn), đánh dấu VIP nếu cần
3. Kéo thả thư mục chứa ảnh hoặc chọn từng file
4. Frontend đọc file từ thư mục bằng `webkitGetAsEntry`, sắp xếp theo tên (numeric)
5. Frontend hiển thị preview ảnh, cho phép sắp xếp lại thứ tự
6. Nhấn "Upload X trang"
7. Frontend gửi FormData `POST /api/comics/:comicId/chapters` (multipart, pages[])
8. Backend upload từng ảnh lên Cloudinary (folder: `metruyen/comics/{slug}/chapter-{n}`)
9. Backend tạo chapter, cập nhật chapter navigation (previousChapter/nextChapter)
10. Chuyển hướng đến danh sách chapter

Ngoại lệ

Tình huống	Xử lý
File không phải ảnh	Multer filter chặn, trả về 400
File > 5MB	Multer limit, trả về 400
Số chapter trùng	Backend trả về 400

Không upload ảnh nào	Backend trả về 400
Không có ảnh bìa	Backend trả về 400

UC10: Quản lý user (Admin)

Mục	Nội dung
ID	UC10
Tên	Quản lý user
Tác nhân chính	Admin
Mô tả	Xem danh sách user, sửa thông tin, đặt lại mật khẩu, xóa user
Tiền điều kiện	Admin đã đăng nhập
Hậu điều kiện	Thông tin user thay đổi trong CSDL

Kịch bản chính

- Admin vào trang quản lý user (`/admin/users`)
- Danh sách user hiển thị với phân trang, có thể tìm kiếm
- Admin có thể:
 - Click "Sửa": thay đổi role, displayName, isVIP, M-Point → PUT `/api/admin/users/:id`
 - Click "Đặt lại mật khẩu": nhập mật khẩu mới → PUT `/api/admin/users/:id/password`
 - Click "Xóa": xác nhận → DELETE `/api/admin/users/:id`

Ngoại lệ

Tình huống	Xử lý
Admin tự xóa chính mình	Backend trả về 400: "You cannot delete your own account"
User không tồn tại	Backend trả về 404

2. Thiết kế hệ thống

2.1. Kiến trúc tổng thể

Hệ thống MeTruyen được thiết kế và xây dựng dựa trên kiến trúc phân tán Client-Server tiêu chuẩn, đảm bảo sự tách biệt rõ ràng giữa luồng xử lý giao diện và logic nghiệp vụ. Mô hình này giúp hệ thống hoạt động ổn định, dễ dàng bảo trì và mở rộng trong tương lai.

- Kiến trúc tổng thể bao gồm ba tầng chính:

- **Tầng Client (Frontend):** Được xây dựng dưới dạng ứng dụng trang đơn (Single Page Application - SPA) sử dụng thư viện React, công cụ Vite và framework Tailwind CSS. Tầng này bao gồm các thành phần giao diện (Components như Layout, ComicCard, CommentSection), các trang chức năng (Pages như Home, ComicDetail, Reader, Admin), và sử dụng thư viện Zustand để quản lý trạng thái xác thực (authStore). Client giao tiếp với Server thông qua các HTTP request sử dụng Axios, truyền tải dữ liệu dưới định dạng JSON và đính kèm Bearer Token (JWT) trong Header để xác thực.
- **Tầng Server (Backend):** Vận hành trên nền tảng Node.js và framework Express.js, đóng vai trò là trung tâm xử lý dữ liệu và cung cấp các RESTful API. Hệ thống API được phân bổ thành nhiều tuyến đường (routes) chuyên biệt như: Auth, Comics, Chapters, Users, Comments, Category và Admin. Mọi request từ Client đều phải đi qua các lớp trung gian (Middleware) như cors, express.json, và errorHandler trước khi được điều phối bởi Controllers, xử lý logic tại Services và tương tác với Models.
- **Tầng Database (Cơ sở dữ liệu):** Toàn bộ dữ liệu của hệ thống được lưu trữ tập trung tại cơ sở dữ liệu MongoDB thông qua 6 collections riêng biệt. Server sẽ giao tiếp với hệ cơ sở dữ liệu này thông qua thư viện ODM Mongoose, giúp định nghĩa schema và thực thi các truy vấn một cách an toàn.

2.2. Mô hình dữ liệu

Hệ thống bao gồm 6 collection chính, sử dụng cơ chế tham chiếu (reference thông qua ObjectId) để tạo mối liên kết giữa các thực thể:

a, Collection User

Mô tả: Lưu trữ thông tin tài khoản người dùng, phân quyền, trạng thái VIP, điểm thưởng và thư viện cá nhân (lịch sử đọc, truyện đã lưu).

Field	Type	Required	Description
_id	ObjectId	Yes	Khóa chính (Unique identifier)
username	String (unique)	Yes	Tên đăng nhập duy nhất
email	String (unique)	Yes	Địa chỉ email duy nhất
password	String	Yes	Mật khẩu người dùng (đã hash)
displayName	String	Yes	Tên hiển thị của người dùng
role	enum['member', 'admin']	Yes	Vai trò của người dùng
avatar	String (URL)	No	Đường dẫn (URL) ảnh đại diện
bio	String	No	Tiêu sử ngắn của người dùng
isVIP	Boolean	Yes	Trạng thái tài khoản VIP
vipExpireDate	Date	No	Ngày hết hạn trạng thái VIP
mPoints	Number	Yes	Số điểm/tiền ảo của người dùng
library.bookmarks	[{ comicId: ref→Comic, addedAt: Date }]	No	Danh sách truyện đã lưu/theo dõi
library.history	[{ comicId: ref→Comic, chapterId: ref→Chapter, lastReadPage: Number, lastReadAt: Date }]	No	Lịch sử đọc truyện
createdAt	Date	Yes	Thời gian tạo tài khoản
updatedAt	Date	Yes	Thời gian cập nhật tài khoản gần nhất

b, Collection Comic

Mô tả: Lưu trữ thông tin tổng quan của một bộ truyện tranh, bao gồm tiêu đề, tác giả, trạng thái phát hành, thống kê lượt xem/theo dõi và các siêu dữ liệu liên quan.

Field	Type	Required	Description
id	ObjectId	Yes	Khóa chính (Unique identifier)
title	String	Yes	Tiêu đề truyện
slug	String (unique)	Yes	Đường dẫn URL thân thiện (duy nhất)
coverImage	{ url: String, publicId: String }	Yes	Thông tin ảnh bìa truyện
description	String	Yes	Tóm tắt nội dung truyện
author	String	Yes	Tên tác giả
categories	[ref→Category]	Yes	Danh sách thể loại (tham chiếu)
tags	[String]	No	Danh sách các thẻ (tags) tìm kiếm
status	enum['ongoing', 'completed','hi atus','cancelled']	Yes	Trạng thái phát hành
publicationYear	Number	No	Năm xuất bản
statistics.totalChapters	Number	Yes	Tổng số chương hiện có
statistics.totalViews	Number	Yes	Tổng lượt xem
statistics.totalBookmarks	Number	Yes	Tổng lượt theo dõi/lưu
isPublished	Boolean	Yes	Trạng thái hiển thị (công khai hay ẩn)
createdBy	ref→User	Yes	Người dùng/Admin đăng truyện
lastChapterUpdate	Date	No	Thời gian cập nhật chương mới nhất
createdAt	Date	Yes	Thời gian tạo truyện
updatedAt	Date	Yes	Thời gian cập nhật truyện gần nhất

c, Collection Chapter

Mô tả: Lưu trữ chi tiết nội dung của từng chương truyện, bao gồm danh sách các trang ảnh, số thứ tự chương, lượt xem và phân quyền truy cập (thường/VIP).

Field	Type	Required	Description
id	ObjectId	Yes	Khóa chính (Unique identifier)
comicId	ref→Comic	Yes	Tham chiếu đến truyện chứa chương này
chapterNumber	Number	Yes	Số thứ tự chương
title	String	No	Tiêu đề của chương (nếu có)
slug	String	Yes	Đường dẫn URL thân thiện của chương
pages	[{ url: String, publicId: String, width: Number, height: Number, order: Number }]	Yes	Danh sách hình ảnh các trang truyện
previousChapter	ref→Chapter	No	Tham chiếu đến chương trước đó
nextChapter	ref→Chapter	No	Tham chiếu đến chương tiếp theo
isPublished	Boolean	Yes	Trạng thái hiển thị công khai
publishDate	Date	Yes	Ngày giờ xuất bản chương
isVIPOnly	Boolean	Yes	Đánh dấu chương chỉ dành cho VIP
views	Number	Yes	Lượt xem của chương
createdAt	Date	Yes	Thời gian tạo chương
updatedAt	Date	Yes	Thời gian cập nhật chương gần nhất

d, Collection Comment

Mô tả: Lưu trữ các bình luận, tương tác của người dùng trên toàn bộ truyện hoặc trên từng chương cụ thể.

Field	Type	Required	Description
_id	ObjectId	Yes	Khóa chính (Unique identifier)
comicId	ref→Comic (indexed)	Yes	Tham chiếu đến truyện được bình luận
userId	ref→User	Yes	Tham chiếu đến người dùng viết bình luận
chapterId	ref→Chapter	No	Tham chiếu đến chương cụ thể (optional)
content	String (max 1000)	Yes	Nội dung bình luận (tối đa 1000 ký tự)
createdAt	Date	Yes	Thời gian tạo bình luận
updatedAt	Date	Yes	Thời gian cập nhật bình luận

e, Collection Payment

Mô tả: Lưu trữ lịch sử giao dịch, thanh toán, nạp điểm hoặc mua các gói VIP của người dùng trên hệ thống.

Field	Type	Required	Description
_id	ObjectId	Yes	Khóa chính (Unique identifier)
userId	ref→User	Yes	Tham chiếu đến người dùng thực hiện giao dịch
transactionId	String (unique)	Yes	Mã giao dịch duy nhất
amount	Number	Yes	Số tiền thanh toán
status	enum['pending','success','failed','refunded']	Yes	Trạng thái giao dịch
paymentMethod	String	Yes	Phương thức thanh toán (vd: Momo, VNPay...)
createdAt	Date	Yes	Thời gian tạo giao dịch
updatedAt	Date	Yes	Thời gian cập nhật giao dịch

f, Collection Category

Mô tả: Lưu trữ danh mục các thể loại truyện tranh, giúp phân loại, sắp xếp và hỗ trợ tính năng lọc/tìm kiếm truyện.

Field	Type	Required	Description
id	ObjectId	Yes	Khóa chính (Unique identifier)
name	String (unique)	Yes	Tên thể loại (duy nhất)
slug	String (unique)	Yes	Đường dẫn URL thân thiện (duy nhất)
description	String	No	Mô tả chi tiết về thể loại
icon	String	No	Biểu tượng (icon) của thể loại
order	Number	No	Thứ tự ưu tiên hiển thị
comicCount	Number	Yes	Tổng số lượng truyện thuộc thể loại này
createdAt	Date	Yes	Thời gian tạo thể loại
updatedAt	Date	Yes	Thời gian cập nhật thể loại

2.3. Môi quan hệ giữa các Collection

- Dù MongoDB là hệ quản trị cơ sở dữ liệu phi cấu trúc (NoSQL) lưu trữ dưới định dạng Document, hệ thống vẫn duy trì tính toàn vẹn và liên kết dữ liệu chặt chẽ. Điều này được thực hiện thông qua hai chiến lược mô hình hóa cốt lõi, giúp xử lý linh hoạt các bài toán quan hệ 1-1, 1-N và N-N:

- **Tham chiếu (Referenced Documents):** Sử dụng ObjectId để liên kết các tài liệu độc lập với nhau (tương tự Khóa ngoại trong SQL), tối ưu cho các dữ liệu dùng chung và thường xuyên cập nhật.
- **Nhúng dữ liệu (Embedded Documents):** Lưu trữ trực tiếp tài liệu con vào bên trong tài liệu cha, giúp tối ưu hóa hiệu năng đọc (Read Performance) khi có thể lấy toàn bộ dữ liệu chỉ với một lần truy vấn duy nhất.

A	B	Kiểu	Field liên kết	Ý nghĩa
User	Comment	1:N	User._id ← Comment.userId	1 user có nhiều bình luận
User	Payment	1:N	User._id ← Payment.userId	1 user có nhiều giao dịch
User	Comic	N:N	User.library.bookmarks[].comicId → Comic. id	User bookmark nhiều truyện
Comic	Chapter	1:N	Comic._id ← Chapter.comicId	1 truyện có nhiều chapter

Comic	Category	N:N	Comic.categories[] → Category._id	1 truyện thuộc nhiều thể loại
Comic	Comment	1:N	Comic._id ← Comment.comicId	1 truyện có nhiều bình luận
Comic	User	N:1	Comic.createdBy → User._id	Nhiều truyện do 1 admin tạo
Chapter	Comment	1:N	Chapter._id ← Comment.chapterId	1 chapter có nhiều bình luận
Chapter	Chapter	1:1	Chapter.previousChapter/nextChapter → Chapter._id	Liên kết chapter trước/sau
Category	Comic	1:N	Category._id ← Comic.categories[]	1 thể loại chứa nhiều truyện

2.4. Danh sách API endpoints

Hệ thống được thiết kế dựa trên chuẩn kiến trúc RESTful API, phân chia thành các nhóm xử lý từng loại tài nguyên độc lập nhằm đảm bảo tính toàn vẹn, tính bảo mật và dễ dàng bảo trì hệ thống. Chi tiết các endpoint được cấu trúc và liệt kê cụ thể theo từng phân nhóm chức năng dưới đây:

- **Nhóm Authentication (Xác thực tài khoản)**
 - POST /api/auth/register: Khởi tạo và cấp phát một tài khoản người dùng mới trên hệ thống.
 - POST /api/auth/login: Xác thực thông tin đăng nhập của người dùng và cấp phát mã mã hóa Token (JWT).
 - GET /api/auth/me: Truy xuất thông tin chi tiết của người dùng hiện tại đang đăng nhập dựa trên Token được đính kèm.
- **Nhóm Comics (Quản lý thông tin truyện tranh)**
 - GET /api/comics: Truy xuất danh sách các bộ truyện tranh hiện có, tích hợp sẵn các tính năng phân trang, tìm kiếm và bộ lọc dữ liệu.
 - GET /api/comics/slug/:slug: Xem thông tin chi tiết của một bộ truyện cụ thể dựa trên đường dẫn tĩnh định danh (slug).
 - GET /api/comics/slug/:slug/chapters: Truy xuất toàn bộ danh sách các chương thuộc một bộ truyện cụ thể thông qua đường dẫn slug.
 - GET /api/comics/:id: Xem thông tin chi tiết của một bộ truyện dựa trên mã định danh ID duy nhất trong cơ sở dữ liệu.

- GET /api/comics/:id/chapters: Lấy danh sách toàn bộ các chương thuộc bộ truyện tương ứng dựa trên ID truyện.
 - POST /api/comics/:id/view: Xử lý tăng tự động biến đếm tổng số lượt xem của bộ truyện mỗi khi có độc giả truy cập.
 - POST /api/comics: Khởi tạo một bộ truyện mới vào hệ thống dữ liệu (chức năng giới hạn, yêu cầu quyền Admin).
 - PUT /api/comics/:id: Cập nhật các thông tin thay đổi của một bộ truyện dựa trên mã ID (chức năng giới hạn, yêu cầu quyền Admin).
 - DELETE /api/comics/:id: Loại bỏ vĩnh viễn một bộ truyện ra khỏi hệ thống dữ liệu dựa trên mã ID (chức năng giới hạn, yêu cầu quyền Admin).
 - POST /api/comics/:comicId/chapters: Khởi tạo một chương truyện mới đính kèm vào bộ truyện cha tương ứng (chức năng giới hạn, yêu cầu quyền Admin).
- **Nhóm Chapters (Quản lý nội dung chương truyện)**
 - GET /api/chapters/:id: Truy xuất nội dung hình ảnh chi tiết của chương truyện, đồng thời kích hoạt cơ chế kiểm tra và chặn truy cập nếu tài khoản chưa đăng nhập hoặc không đủ quyền hạn đối với chương VIP.
 - PUT /api/chapters/:id: Chỉnh sửa tiêu đề, số thứ tự hoặc cập nhật các trang ảnh của chương truyện dựa trên ID (chức năng giới hạn, yêu cầu quyền Admin).
 - DELETE /api/chapters/:id: Xóa bỏ hoàn toàn một chương truyện ra khỏi hệ thống dựa trên ID chương (chức năng giới hạn, yêu cầu quyền Admin).
- **Nhóm Categories (Quản lý danh mục thể loại)**
 - GET /api/categories: Lấy toàn bộ danh sách các thể loại truyện tranh hiện có trên hệ thống để hiển thị cho người dùng.
 - POST /api/categories: Khởi tạo thêm một thể loại truyện mới vào kho dữ liệu phân loại (chức năng giới hạn, yêu cầu quyền Admin).
 - PUT /api/categories/:id: Thay đổi tên gọi, slug định danh hoặc mô tả chi tiết của thể loại dựa trên ID (chức năng giới hạn, yêu cầu quyền Admin).
 - DELETE /api/categories/:id: Loại bỏ một thể loại ra khỏi danh mục phân loại dựa trên ID (chức năng giới hạn, yêu cầu quyền Admin).

- **Nhóm Comments (Quản lý tương tác bình luận - được gắn kết tại /api/comics/:comicId/comments)**
 - GET /: Truy xuất danh sách các bình luận thuộc bộ truyện, hỗ trợ cơ chế phân trang và cho phép truyền tham số lọc riêng theo chương cụ thể (chapterId).
 - POST /: Gửi và lưu trữ một bình luận mới từ phía người đọc vào bộ truyện hoặc chương truyện tương ứng (yêu cầu xác thực Token thành viên).
 - DELETE /:commentId: Tiến hành gỡ bỏ một nội dung bình luận ra khỏi hệ thống dựa trên mã bình luận (yêu cầu xác thực Token thành viên sở hữu hoặc quyền Admin).
- **Nhóm Users (Quản lý thông tin tài khoản và thư viện tiện ích)**
 - GET /api/users/profile: Lấy toàn bộ thông tin chi tiết thuộc hồ sơ cá nhân của tài khoản đang kết nối (yêu cầu xác thực Token).
 - PUT /api/users/profile: Cập nhật các thông tin thay đổi trong hồ sơ cá nhân bao gồm tên hiển thị, bio và ảnh đại diện tải lên Cloudinary (yêu cầu xác thực Token).
 - PUT /api/users/password: Thực hiện quy trình kiểm tra và thiết lập mật khẩu bảo mật mới cho tài khoản (yêu cầu xác thực Token).
 - GET /api/users/library: Truy xuất toàn bộ kho lưu trữ cá nhân của thành viên, bao gồm danh sách các bộ truyện theo dõi và danh mục lịch sử đọc (yêu cầu xác thực Token).
 - POST /api/users/library/bookmark/:comicId: Xử lý chuyển đổi trạng thái (bật/tắt) việc lưu một bộ truyện vào danh sách yêu thích cá nhân dựa trên ID truyện (yêu cầu xác thực Token).
 - POST /api/users/library/history/:chapterId: Tự động ghi nhận hoặc cập nhật dấu mốc lịch sử đọc của người dùng đối với chương truyện tương ứng (yêu cầu xác thực Token).
 - POST /api/users/upgrade-vip: Tiếp nhận yêu cầu chuyển đổi và trừ điểm tích lũy để nâng cấp trạng thái tài khoản lên hạng VIP (yêu cầu xác thực Token).
 - POST /api/users/recharge-mpoints: Khởi tạo và gửi yêu cầu nạp điểm số ảo M-Point vào ví điện tử cá nhân (yêu cầu xác thực Token).

- **Nhóm Admin (Phân hệ công cụ quản trị chuyên sâu)**

- GET /api/admin/stats: Lấy toàn bộ số liệu dữ liệu tổng hợp (thống kê lượng người dùng, truyện, lượt xem) phục vụ cho giao diện biểu đồ Dashboard.
- GET /api/admin/payments: Truy xuất toàn bộ danh sách dữ liệu lịch sử các giao dịch tài chính nạp điểm phát sinh trên hệ thống.
- PUT /api/admin/payments/:id: Thực hiện kiểm tra, phê duyệt hoặc cập nhật trạng thái xử lý cho một yêu cầu giao dịch nạp điểm cụ thể.
- DELETE /api/admin/payments/:id: Xóa bản ghi thông tin của một giao dịch tài chính ra khỏi hệ thống quản lý.
- GET /api/admin/users: Lấy danh sách toàn bộ các tài khoản người dùng đã đăng ký thành viên trên nền tảng.
- PUT /api/admin/users/:id: Thay đổi các thông tin cấu hình, thông tin cá nhân hoặc điều chỉnh phân cấp quyền hạn đối với một tài khoản cụ thể.
- PUT /api/admin/users/:id/password: Thực hiện thay đổi bắt buộc, đặt lại mật khẩu đăng nhập mới cho một tài khoản người dùng.
- DELETE /api/admin/users/:id: Xóa bỏ vĩnh viễn và đóng tài khoản người dùng ra khỏi hệ thống cơ sở dữ liệu.

2.5 Cấu trúc mã nguồn

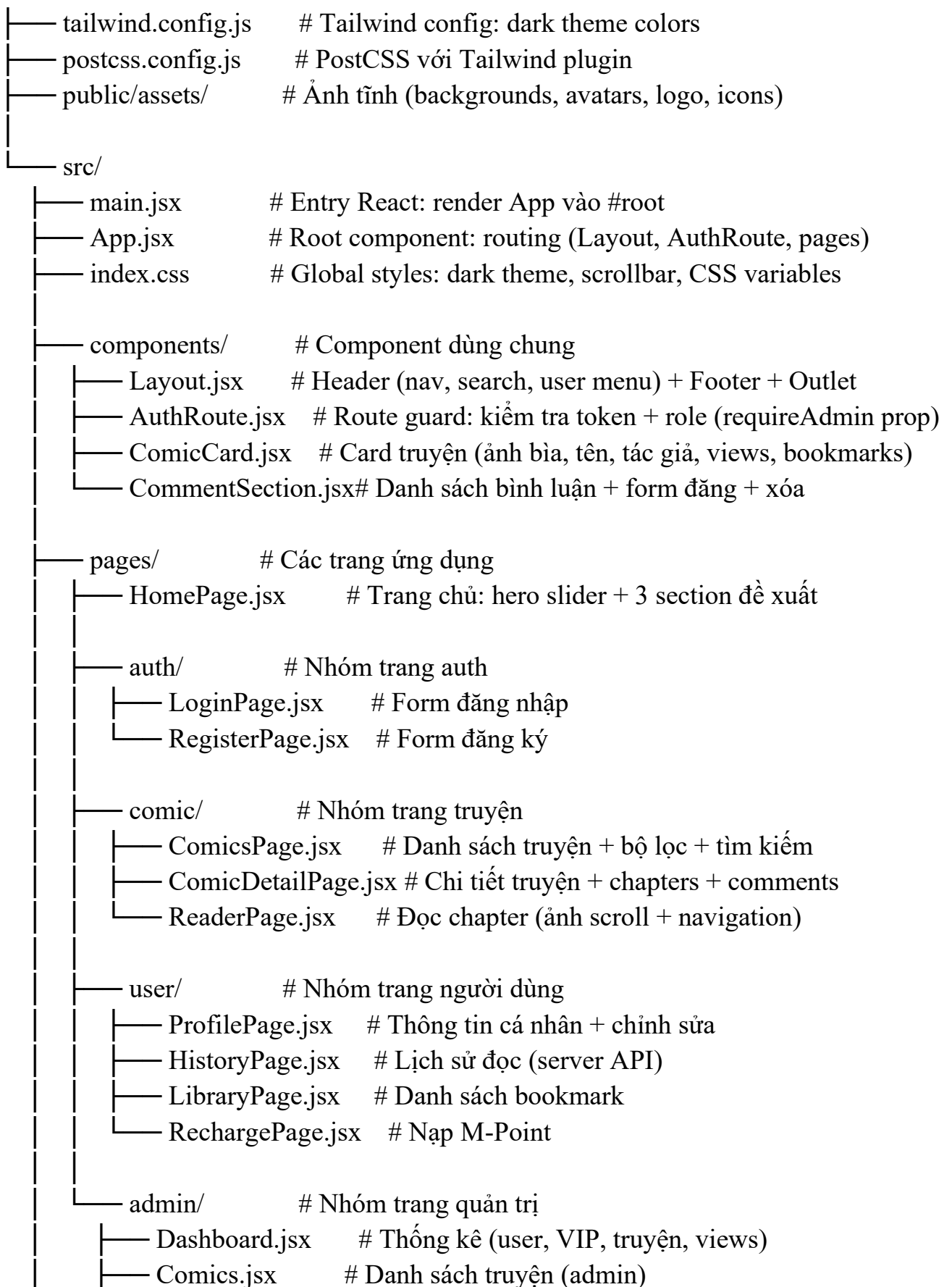
- Cấu trúc Backend

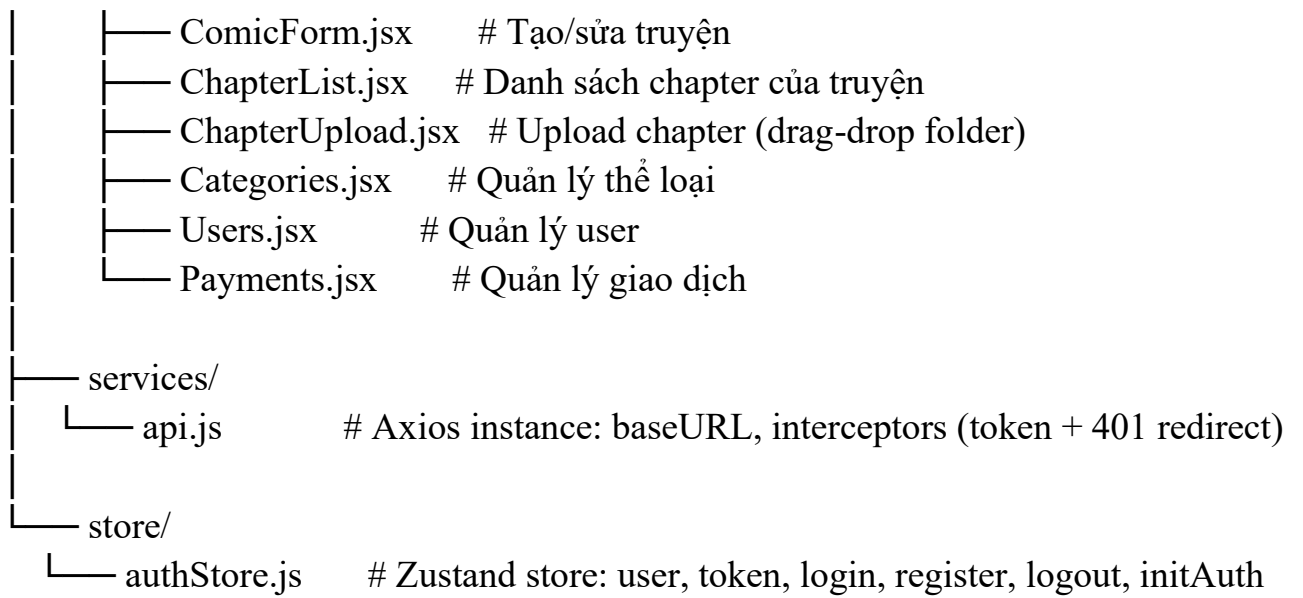
```
backend/
├── .env                # Biến môi trường
├── package.json        # Dependencies và scripts
├── src/
│   ├── server.js       # Entry point: khởi tạo Express, mount routes, middleware
│   ├── config/
│   │   ├── database.js # Kết nối MongoDB (Mongoose)
│   │   └── clouinary.js # Cấu hình upload/delete Clouinary
│   ├── models/         # Mongoose schemas (6 collections)
│   │   ├── User.js     # User: username, email, password, role, isVIP, mPoints,library
│   │   ├── Comic.js    # Comic: title, slug, coverImage, categories, statistics
│   │   └── Chapter.js  # Chapter: comicId, chapterNumber, pages[], navigation
```

- ├── Category.js # Category: name, slug, comicCount
- ├── Comment.js # Comment: comicId, userId, chapterId, content
- ├── Payment.js # Payment: userId, transactionId, amount, status
- ├── controllers/ # Xử lý logic nghiệp vụ
 - ├── authController.js # Register, Login, GetMe
 - ├── comicController.js # CRUD comic + view increment
 - ├── chapterController.js # CRUD chapter + navigation + page upload
 - ├── categoryController.js # CRUD category
 - ├── commentController.js # Create, Delete comment
 - ├── userController.js # Profile, Library, Bookmark, History, VIP, Payment
- ├── routes/ # Định tuyến REST API
 - ├── authRoutes.js # /api/auth (register, login, me)
 - ├── comicRoutes.js # /api/comics (CRUD, chapters, view)
 - ├── chapterRoutes.js # /api/chapters (read, update, delete)
 - ├── categoryRoutes.js # /api/categories (CRUD)
 - ├── commentRoutes.js # /api/comics/:id/comments
 - ├── userRoutes.js # /api/users (profile, library, history, payment)
 - ├── adminRoutes.js # /api/admin (stats, users, payments)
- ├── middlewares/ # Middleware xử lý request
 - ├── auth.js # verifyToken, isAdmin, protect
 - ├── errorHandler.js # AppError class + error handler
 - ├── upload.js # Multer: memoryStorage, fileFilter, limits
- ├── utils/
 - ├── asyncHandler.js # Wrap async function, catch error → next(err)
 - ├── jwt.js # generateToken, verifyToken (jsonwebtoken)

- Cấu trúc Frontend

```
frontend/
├── .env # VITE_API_BASE_URL=/api
├── index.html # HTML entry point
├── vite.config.js # Vite config: port 3000, proxy /api → localhost:5000
```





CHƯƠNG 4: KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM

1 Môi trường cài đặt

- Phần cứng và hệ điều hành:

- CPU: Intel Core i5, RAM 8GB
- Hệ điều hành: Windows 11

- Phần mềm:

- Node.js v20.x
- MongoDB 7.0 (MongoDB Atlas cho cloud database)
- Git (quản lý phiên bản)
- Visual Studio Code (IDE)
- Postman (kiểm thử API)

- Thông số kỹ thuật:

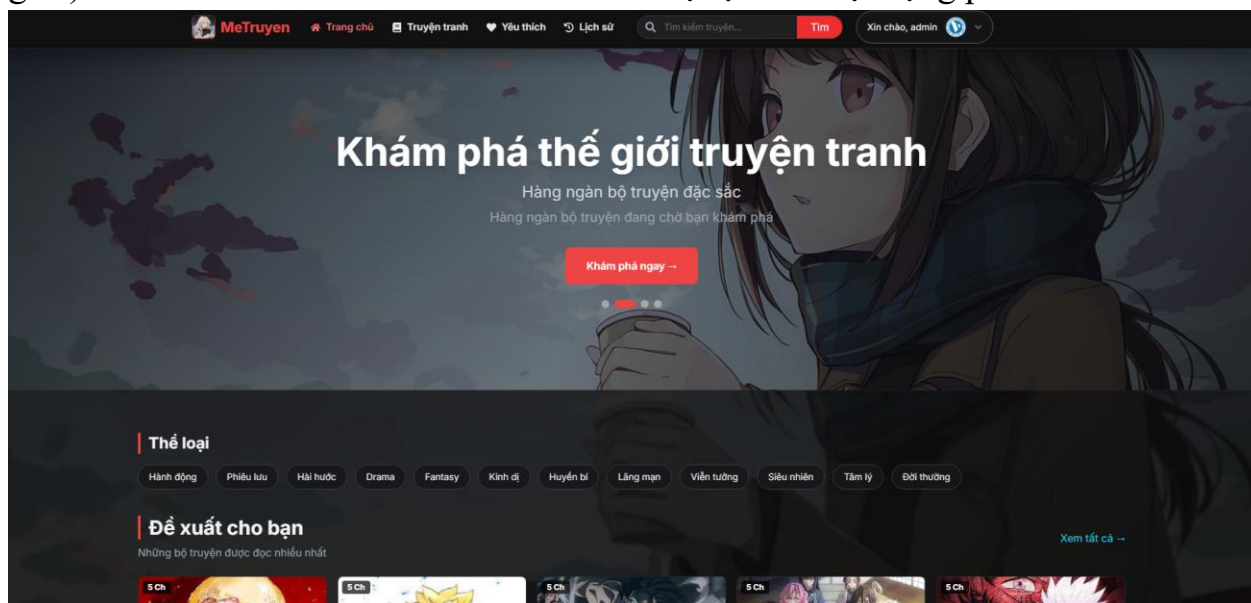
- Backend: Express.js chạy port 5000
- Frontend: Vite dev server chạy port 3000 (proxy API sang 5000)
- Database: MongoDB Atlas cluster (free tier)

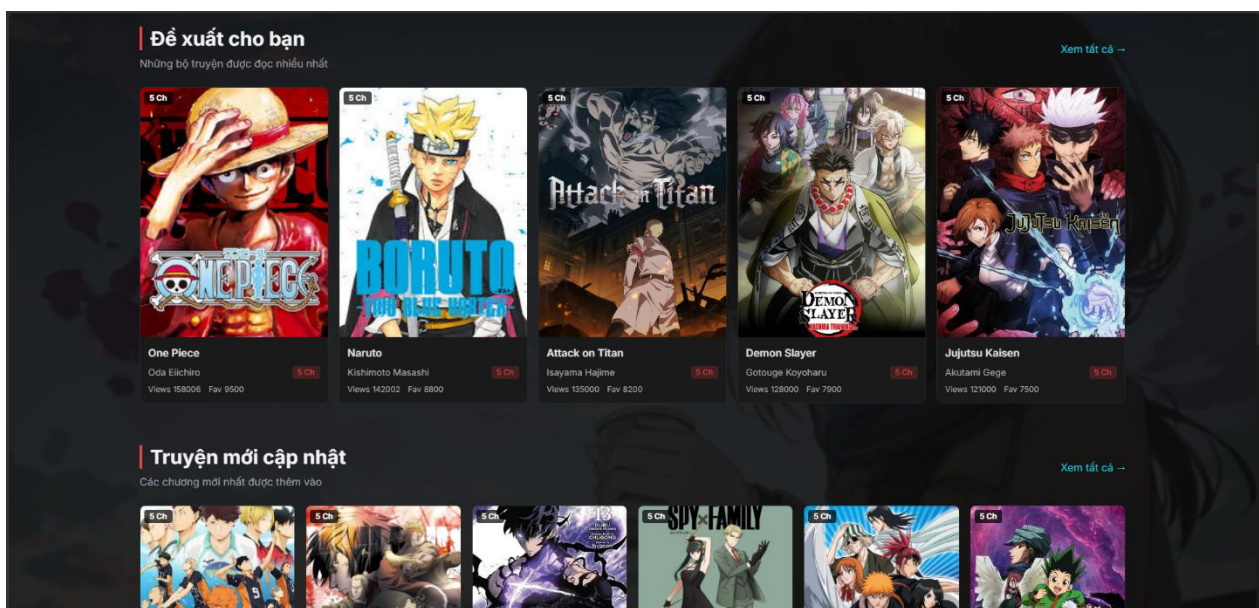
2 Kết quả và kiểm thử

2.1. Kết quả

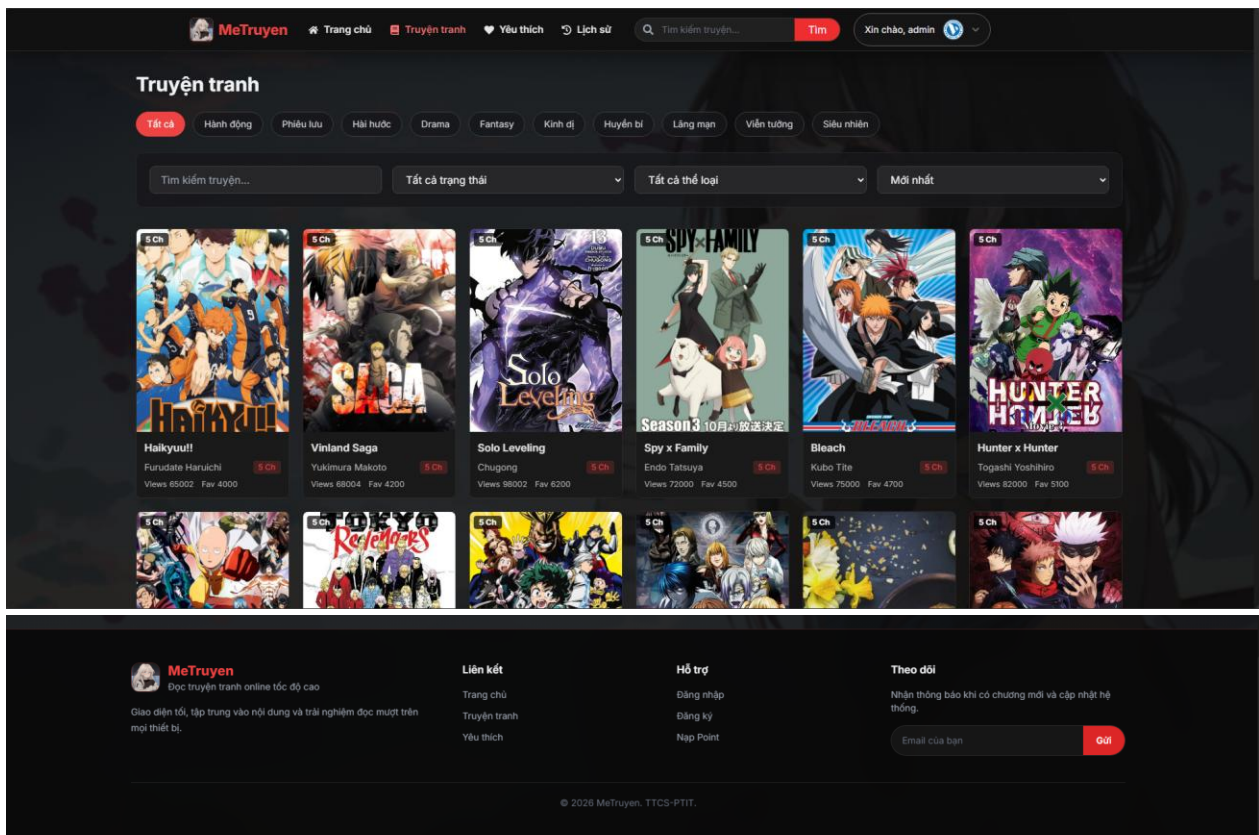
Giao diện hệ thống hiện ra gồm các trang chính:

1. Trang chủ (HomePage): Slider hero với 4 ảnh nền, chuyển đổi tự động. Ba section nội dung: Đề xuất (5 card grid), Truyện mới (6 card grid), Yêu thích (6 card grid). Mỗi section có link "Xem tất cả →". Bộ lọc thể loại dạng pill buttons.

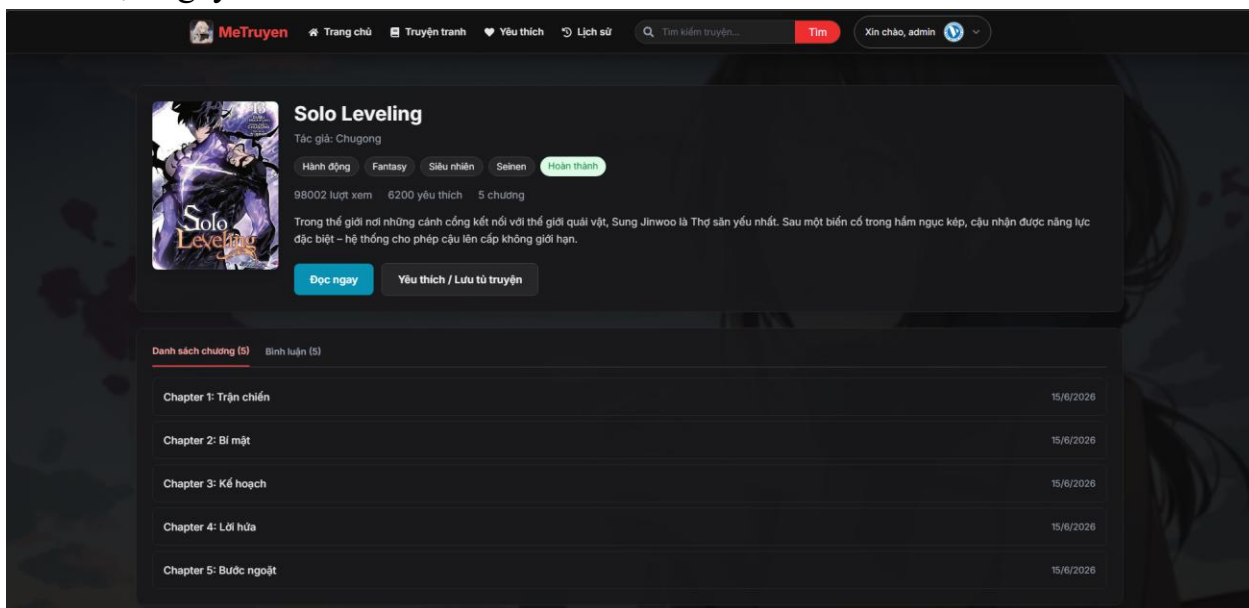




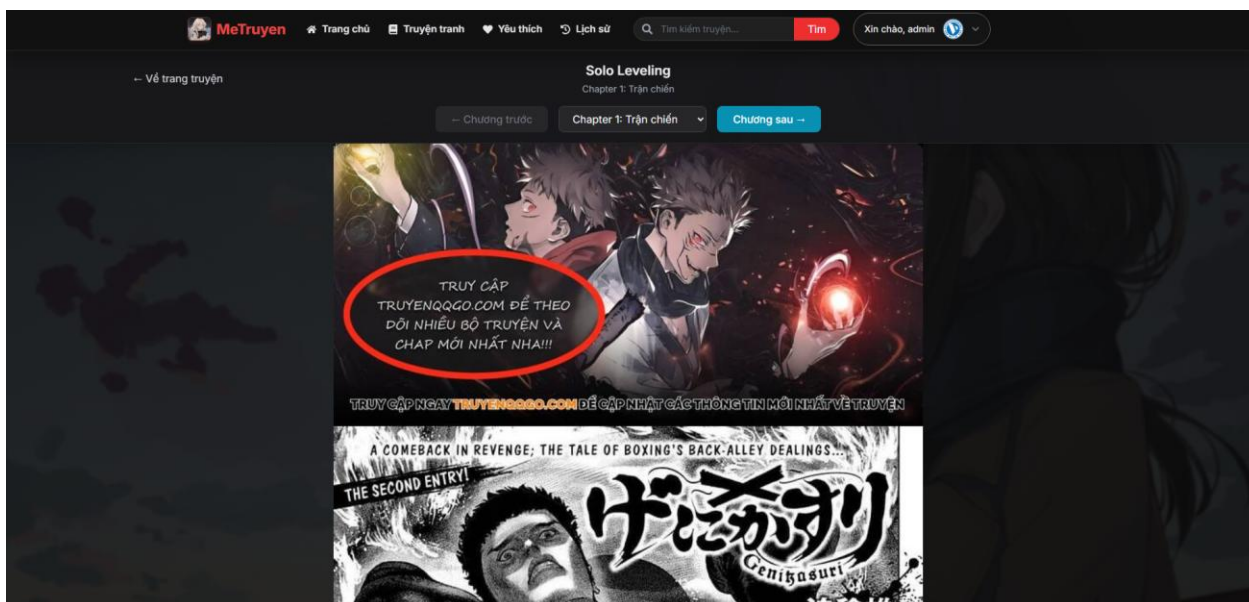
2. Trang danh sách truyện (ComicsPage): Thanh tìm kiếm, bộ lọc (thể loại, trạng thái, sắp xếp). Grid danh sách truyện với phân trang.



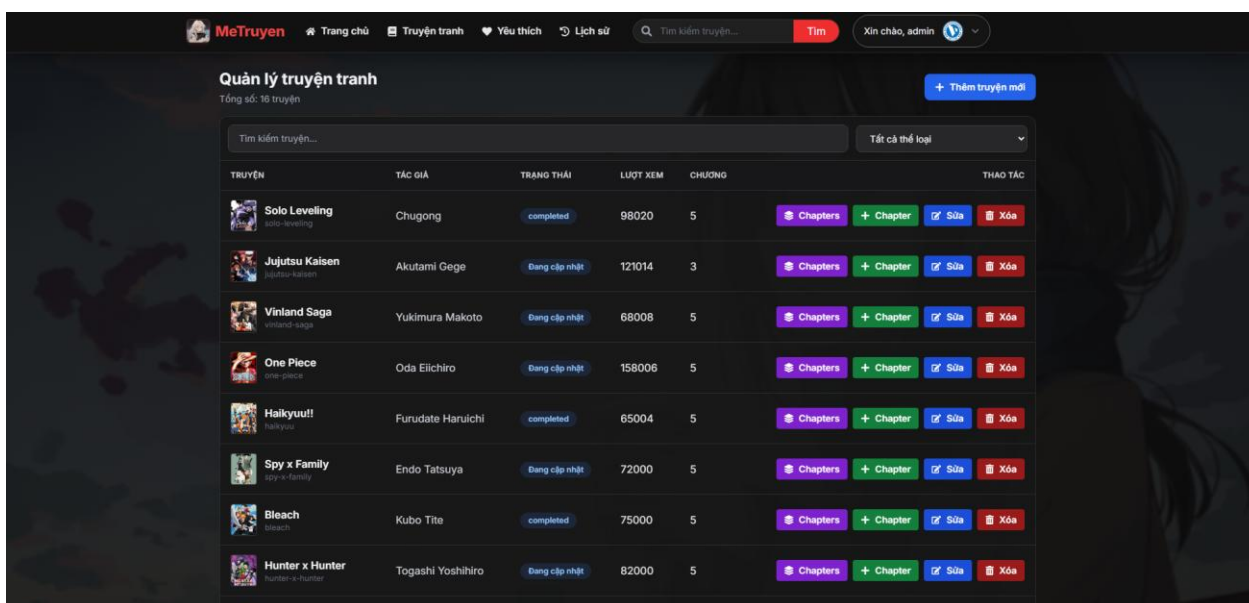
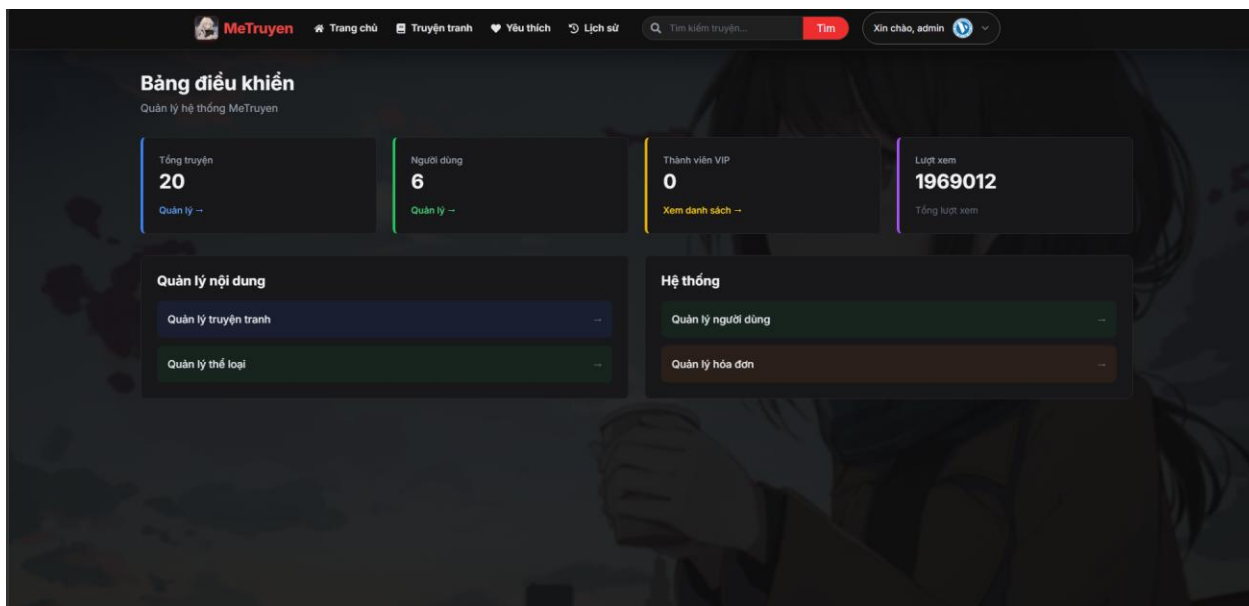
3. Trang chi tiết truyện (ComicDetailPage): Ảnh bìa, thông tin truyện (tác giả, thể loại, trạng thái, view, bookmark). Hai tab: "Danh sách chương" và "Bình luận". Nút "Đọc ngay" và "Yêu thích".



4. Trang đọc (ReaderPage): Yêu cầu đăng nhập. Hiện thị ảnh chapter, điều hướng chapter trước/sau. Phân bình luận chapter ở cuối trang. Kiểm tra VIP, hiển thị thông báo lỗi nếu không có quyền.



5. Trang quản trị (Admin): Dashboard thống kê, quản lý truyện (CRUD), upload chapter (drag-drop folder), quản lý user, thẻ loại, thanh toán.



Quản lý người dùng
Tổng số: 6 người dùng

Tim kiếm: Lọc theo vai trò: Tất cả Làm mới

NGƯỜI DÙNG	EMAIL	VAI TRÒ	M-POINT	VIP	NGÀY TẠO	THAO TÁC
admin123 @admin123	admin123@gmail.com	User	\$ 0	Không	15/6/2026	Sửa Đổi MK Xóa
Hoàng Long @hoanglong	hoanglong@metruyen.com	User	\$ 30000	Không	15/6/2026	Sửa Đổi MK Xóa
Minh Anh @minhanh	minhanh@metruyen.com	User	\$ 10000	Không	15/6/2026	Sửa Đổi MK Xóa
Minh Đức @reader	reader@metruyen.com	User	\$ 20000	Không	15/6/2026	Sửa Đổi MK Xóa
Nguyễn Thành Nam @demo	demo@metruyen.com	User	\$ 50000	Không	15/6/2026	Sửa Đổi MK Xóa
Admin MeTruyen @admin	admin@gmail.com	Admin	\$ 949999	VIP 22/6/2026	15/6/2026	Sửa Đổi MK Xóa

Quản lý hóa đơn
Tổng: 10 giao dịch - Thành công trên trang này: 430,000 VND

Tim kiếm: Tất cả trạng thái

Mã giao dịch	Người dùng	Số tiền	Phương thức	Trạng thái	Thời gian	Hành động
TX037545299	reader reader@metruyen.com	50,000 VND	ZaloPay	Thất bại	21:12:34 14/6/2026	Sửa Xóa
TX037545288	demo demo@metruyen.com	100,000 VND	Chuyển khoản	Thất bại	21:12:34 13/6/2026	Sửa Xóa
TX037545277	hoanglong hoanglong@metruyen.com	50,000 VND	Chuyển khoản	Đang chờ	21:12:34 12/6/2026	Sửa Xóa
TX037545256	minhanh minhanh@metruyen.com	10,000 VND	ZaloPay	Đang chờ	21:12:34 11/6/2026	Sửa Xóa
TX037545245	reader reader@metruyen.com	50,000 VND	viettelpay	Thành công	21:12:34 10/6/2026	Sửa Xóa
TX037545234	demo demo@metruyen.com	200,000 VND	ZaloPay	Thành công	21:12:34 9/6/2026	Sửa Xóa
TX037545213	hoanglong hoanglong@metruyen.com	100,000 VND	viettelpay	Thành công	21:12:34 8/6/2026	Sửa Xóa
TX037545282	minhanh minhanh@metruyen.com	50,000 VND	Chuyển khoản	Thành công	21:12:34 7/6/2026	Sửa Xóa
TX037545191	reader reader@metruyen.com	20,000 VND	ZaloPay	Thành công	21:12:34 6/6/2026	Sửa Xóa

=> Web hoạt động ổn định, fetch dữ liệu từ server và cloudinary về và phản hồi nhanh cho hầu hết giao diện.

2.2. Kiểm thử

2.2.1. Nhóm chức năng Guest (khách chưa đăng nhập)

STT	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả
1	Xem trang chủ	Truy cập '/'	Hiển thị trang chủ với hero slider, 3 section truyện, bộ lọc thể loại	Đạt

2	Xem danh sách truyện	Truy cập `/comics`	Hiển thị grid danh sách truyện kèm phân trang	Đạt
3	Lọc truyện theo thể loại	Click thể loại "Hành động"	Danh sách chỉ hiển thị truyện thể loại Hành động	Đạt
4	Tìm kiếm truyện	Nhập "One Piece" vào ô tìm kiếm	Hiển thị kết quả tìm kiếm chứa "One Piece"	Đạt
5	Xem chi tiết truyện	Click vào truyện bất kỳ	Hiển thị thông tin truyện + danh sách chapter + bình luận	Đạt
6	Xem bình luận	Vào tab "Bình luận" ở trang chi tiết truyện	Hiển thị danh sách bình luận kèm phân trang	Đạt
7	Đăng ký tài khoản	Nhập username, email, password hợp lệ	Tạo tài khoản thành công, thông báo + chuyển đến `/login`	Đạt
8	Đăng ký với email trùng	Nhập email đã tồn tại	Báo lỗi: "Email hoặc username đã được sử dụng"	Đạt
9	Đăng ký với username trùng	Nhập username đã tồn tại	Báo lỗi: "Email hoặc username đã được sử dụng"	Đạt
10	Đăng ký với password < 6 ký tự	Nhập password "123"	Báo lỗi validation từ Mongoose	Đạt
11	Đăng ký với email sai định dạng	Nhập email "abc"	Báo lỗi "Email không hợp lệ"	Đạt
12	Đăng nhập	Nhập email + password hợp lệ	Đăng nhập thành công, lưu token → chuyển về trang chủ	Đạt
13	Đăng nhập sai mật khẩu	Nhập đúng email, sai password	Báo lỗi: "Email hoặc mật khẩu không đúng"	Đạt

14	Đăng nhập với email không tồn tại	Nhập email chưa đăng ký	Báo lỗi: "Email hoặc mật khẩu không đúng"	Đạt
15	Truy cập trang yêu cầu auth	Truy cập `/history` khi chưa đăng nhập	Redirect đến `/login` (AuthRoute)	Đạt

2.2.2. Nhóm chức năng User (đã đăng nhập)

STT	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả
16	Đọc chapter	Click chapter bất kỳ	Hiển thị ảnh chapter dạng scroll, có navigation chapter trước/sau	Đạt
17	Đọc chapter không tồn tại	Truy cập `/read/abc123`	Báo lỗi "Không thể tải chương"	Đạt
18	Đọc chapter VIP (không có VIP)	Click chapter có badge VIP	Báo lỗi "Chapter này yêu cầu tài khoản VIP" (403)	Đạt
19	Lịch sử đọc (sau khi đọc chapter)	Đọc 1 chapter → vào `/history`	Hiển thị chapter vừa đọc trong lịch sử	Đạt
20	Bookmark truyện	Click nút "Yêu thích" ở trang chi tiết	Trái tim đổi màu, số bookmark tăng 1	Đạt
21	Bỏ bookmark	Click lại nút "Yêu thích"	Trái tim trở lại màu cũ, số bookmark giảm 1	Đạt
22	Xem danh sách bookmark	Vào `/library`	Hiển thị danh sách truyện đã bookmark	Đạt

23	Đăng bình luận	Nhập nội dung + gửi ở trang chi tiết truyện	Bình luận mới xuất hiện trong danh sách	Đạt
24	Đăng bình luận ở chapter	Nhập nội dung + gửi ở trang đọc chapter	Bình luận xuất hiện kèm chapterId	Đạt
25	Đăng bình luận với nội dung rỗng	Không nhập nội dung, nhấn gửi	Frontend không cho gửi	Đạt
26	Xóa bình luận của mình	Hover vào bình luận → click Xóa	Bình luận biến mất khỏi danh sách	Đạt
27	Xóa bình luận của người khác	Thử xóa bình luận không phải của mình	Backend trả về 403, không xóa được	Đạt
28	Xem profile	Vào `/profile`	Hiển thị thông tin user (username, email, displayName, mPoints)	Đạt
29	Sửa displayName	Nhập displayName mới → lưu	DisplayName thay đổi, hiển thị tên mới	Đạt
30	Sửa bio	Nhập bio mới → lưu	Bio thay đổi	Đạt
31	Đổi mật khẩu	Nhập current password + new password hợp lệ	Đổi mật khẩu thành công, đăng nhập lại được với mật khẩu mới	Đạt
32	Đổi mật khẩu sai current password	Nhập sai current password	Báo lỗi: "Current password is incorrect"	Đạt

33	Nạp M-Point	Nhập số tiền, mã giao dịch, phương thức → nạp	Số dư M-Point tăng, giao dịch được tạo	Đạt
34	Nạp M-Point với mã trùng	Nhập mã giao dịch đã dùng	Báo lỗi: "Mã giao dịch đã được sử dụng"	Đạt
35	Nạp M-Point với số tiền ≤ 0	Nhập amount = 0	Báo lỗi: "Số tiền không hợp lệ"	Đạt
36	Nâng cấp VIP (đủ M-Point)	Chọn gói 7 ngày (50k)	VIP được kích hoạt, số dư giảm 50k	Đạt
37	Nâng cấp VIP (không đủ M-Point)	Chọn gói 365 ngày (1tr) khi không đủ	Báo lỗi: "Không đủ M-Point"	Đạt
38	Nâng cấp VIP gia hạn	Mua thêm VIP khi đang còn hạn	Hạn VIP được cộng dồn (không reset)	Đạt
39	Đọc chapter VIP (có VIP)	Click chapter VIP sau khi nâng cấp	Hiển thị ảnh chapter bình thường	Đạt

2.2.3. Nhóm chức năng Admin

STT	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả
40	Xem Dashboard	Vào `/admin`	Hiển thị thông kê: tổng user, VIP, truyện, lượt xem	Đạt
41	Xem danh sách truyện (admin)	Vào `/admin/comics`	Hiển thị danh sách truyện kèm nút thêm/sửa/xóa	Đạt

42	Tạo truyện mới	Nhập thông tin + upload ảnh bìa → tạo	Truyện mới xuất hiện trong danh sách, ảnh bìa upload lên Cloudinary	Đạt
43	Tạo truyện không có ảnh bìa	Không chọn ảnh → tạo	Báo lỗi: yêu cầu upload ảnh bìa	Đạt
44	Sửa truyện	Sửa tên/tác giả/mô tả → lưu	Thông tin truyện thay đổi	Đạt
45	Xóa truyện	Click xóa → xác nhận	Truyện biến mất khỏi danh sách, comicCount category giảm	Đạt
46	Upload chapter mới	Kéo thả folder ảnh → nhập số chapter → upload	Chapter mới được tạo, ảnh upload lên Cloudinary	Đạt
47	Upload chapter trùng số	Nhập số chapter đã tồn tại	Báo lỗi: "Chapter number already exists for this comic"	Đạt
48	Sửa chapter (đổi title + VIP)	Sửa title + bật VIP → lưu	Title thay đổi, chapter hiển thị badge VIP	Đạt
49	Sửa chapter (thay ảnh)	Xóa ảnh cũ + kéo ảnh mới → lưu	Ảnh mới hiển thị, ảnh cũ xóa khỏi Cloudinary	Đạt
50	Xóa chapter	Click xóa chapter	Chapter biến mất, navigation chapter trước/sau cập nhật	Đạt
51	Quản lý thể loại (thêm)	Nhập tên thể loại mới → thêm	Thể loại mới xuất hiện trong danh sách	Đạt
52	Quản lý thể loại (sửa)	Sửa tên thể loại → lưu	Tên thể loại thay đổi	Đạt
53	Quản lý thể loại (xóa)	Click xóa thể loại	Thể loại biến mất	Đạt

54	Xem danh sách user	Vào '/admin/users'	Hiển thị danh sách user kèm phân trang, tìm kiếm	Đạt
55	Sửa user (role, VIP, M-Point)	Sửa user thường → thành admin	User có quyền admin	Đạt
56	Đặt lại mật khẩu user	Nhập mật khẩu mới → lưu	User đăng nhập được với mật khẩu mới	Đạt
57	Xóa user	Click xóa user ≠ chính mình	User biến mất khỏi danh sách	Đạt
58	Xóa chính mình (admin tự xóa)	Admin thử xóa tài khoản của mình	Báo lỗi: "You cannot delete your own account"	Đạt
59	Xem danh sách giao dịch	Vào '/admin/payments'	Hiển thị danh sách giao dịch nạp điểm	Đạt
60	Duyệt giao dịch (đổi status)	Đổi trạng thái từ pending → success	Trạng thái thay đổi	Đạt

2.2.4. Nhóm kiểm thử lỗi và ngoại lệ

STT	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả
61	404 endpoint không tồn tại	GET '/api/xyz'	Trả về 404 JSON (không crash server)	Đạt
62	Request không có token	Gọi API yêu cầu auth không kèm header	Trả về 401: "Access denied. No token provided."	Đạt

63	Token hết hạn	Gửi token cũ đã hết hạn	Trả về 401: "Token invalid or expired." → redirect `/login`	Đạt
64	Upload file không phải ảnh	Upload file .txt vào chapter	Multer chặn, trả về 400: "Only image files are allowed"	Đạt
65	Upload file > 5MB	Upload ảnh 10MB	Multer chặn, trả về 400	Đạt
66	CastError (ObjectId sai)	GET `/api/chapters/abc`	Backend bắt CastError → 404	Đạt
67	Validation Error	Tạo comic không có title	Mongoose validation → 400	Đạt

3. Đánh giá kết quả

- Ưu điểm:

- **Về mặt hiệu năng và độ ổn định:** Hệ thống vận hành mượt mà, thời gian phản hồi (response time) cho phần lớn các API đều đạt mức tối ưu dưới 200ms, đảm bảo trải nghiệm xuyên suốt cho người dùng.
- **Về kiến trúc xử lý:** Áp dụng thành công cơ chế xử lý lỗi (error handling) đồng bộ và tập trung. Điều này giúp hệ thống luôn trả về các thông báo lỗi theo định dạng chuẩn, đảm bảo máy chủ (server) không gặp tình trạng ngưng trệ (crash) khi có ngoại lệ phát sinh.
- **Về trải nghiệm giao diện (UI/UX):** Giao diện nền tối (dark theme) được thiết kế đồng nhất, tích hợp khả năng hiển thị tương thích linh hoạt (responsive) trên cả nền tảng thiết bị di động (mobile) và máy tính cá nhân (desktop).
- **Về tính năng bảo mật:** Nền tảng đã triển khai các tiêu chuẩn bảo mật cơ bản nhưng hiệu quả, bao gồm việc xác thực qua chuẩn JSON Web Token (JWT) và thiết lập cơ chế phân quyền truy cập chặt chẽ dựa trên vai trò (Role-based Access Control).

- **Về tiện ích quản trị:** Công cụ tải lên chương truyện (upload chapter) được tự động hóa cao với khả năng hỗ trợ thao tác kéo-thả (drag-drop) nguyên một thư mục hình ảnh thông qua hàm webkitGetAsEntry, giúp tiết kiệm đáng kể thời gian thao tác cho Quản trị viên.

- Hạn chế/chưa triển khai:

Bên cạnh những kết quả tích cực, dự án vẫn còn tồn tại một số hạn chế về mặt kỹ thuật và trải nghiệm:

- **Chưa hoàn thiện tính năng tương tác trực tiếp:** Hệ thống bình luận hiện tại chưa hỗ trợ đồng bộ hóa dữ liệu theo thời gian thực (real-time) thông qua Socket.io, người dùng vẫn phải tải lại trang để xem các bình luận mới nhất.
- **Chưa hoàn thiện luồng quản lý tài khoản:** Ứng dụng hiện chưa tích hợp chức năng khôi phục mật khẩu (forgot password) qua email, gây bất tiện cho người dùng trong trường hợp quên thông tin đăng nhập.
- **Khả năng tương thích trình duyệt chưa toàn diện:** Tính năng tiện ích kéo-thả thư mục (drag-drop folder) hiện tại mới chỉ hỗ trợ và hoạt động ổn định trên các trình duyệt sử dụng lõi Chromium.
- **Hạn chế trong quản lý tài nguyên:** Hệ thống chưa cung cấp công cụ để chỉnh sửa (edit) trực tiếp hình ảnh sau khi tải lên, đồng thời các API và tài nguyên ảnh cũng chưa được áp dụng cơ chế bộ nhớ đệm (caching) để tối ưu hóa băng thông.

CHƯƠNG 5: KẾT LUẬN

1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và phát triển, nhóm thực hiện đã hoàn thiện nền tảng đọc truyện tranh trực tuyến MeTruyen và đạt được những kết quả kỹ thuật cốt lõi như sau:

- Xây dựng thành công hệ thống Backend hoàn chỉnh, bao gồm 7 nhóm định tuyến (route groups) và hơn 30 điểm cuối API (API endpoints), kết hợp với cơ chế xử lý lỗi tập trung để đảm bảo tính ổn định của máy chủ.
- Phát triển giao diện Frontend theo kiến trúc ứng dụng trang đơn (React SPA) với quy mô hơn 15 trang chức năng, tích hợp hệ thống định tuyến phân quyền (routing) và quản lý trạng thái (state management) hiệu quả thông qua thư viện Zustand.
- Thiết kế hệ quản trị cơ sở dữ liệu MongoDB tối ưu với 6 tập hợp (collection) chuyên biệt, đồng thời áp dụng các kỹ thuật đánh chỉ mục (indexing) phù hợp nhằm tăng tốc độ truy vấn dữ liệu.
- Triển khai chuẩn mực hệ thống bảo mật bằng chuẩn xác thực JWT và thiết lập cơ chế phân quyền truy cập chặt chẽ theo vai trò người dùng (Thành viên và Quản trị viên).
- Hoàn thiện tính năng tương tác qua hệ thống bình luận đa cấp độ (tại trang chi tiết truyện và trang đọc từng chương), có hỗ trợ cơ chế phân trang để tối ưu hóa hiển thị.
- Tích hợp giải pháp tải lên chương truyện (upload chapter) thông qua thao tác kéo-thả (drag-drop) nguyên một thư mục hình ảnh và tự động lưu trữ trên nền tảng đám mây Cloudinary.
- Khởi tạo thành công bộ dữ liệu mẫu (seed data) phong phú để kiểm thử hệ thống, bao gồm: 3 tài khoản người dùng, 16 phân loại thể loại, 20 bộ truyện, 100 chương và 80 bình luận mô phỏng thực tế.
- Biên soạn đầy đủ bộ tài liệu kỹ thuật, bao gồm hướng dẫn cài đặt chi tiết và tài liệu tham chiếu API, phục vụ đắc lực cho công tác vận hành và bảo trì.

2. Hạn chế

Mặc dù đã đạt được những mục tiêu quan trọng, hệ thống hiện tại vẫn còn tồn tại một số hạn chế nhất định cần được khắc phục:

- Mật khẩu của người dùng hiện đang được lưu trữ dưới dạng văn bản thuần túy (plaintext); đây là sự giản lược nhằm phục vụ mục đích trình diễn (demo) nhanh chóng và chưa đáp ứng tiêu chuẩn an toàn cho môi trường thực tế (production).
- Hệ thống chưa cung cấp luồng chức năng hỗ trợ người dùng khôi phục hoặc đặt lại mật khẩu (forgot password / reset password) khi thất lạc thông tin đăng nhập.
- Tính năng bình luận chưa được đồng bộ hóa theo thời gian thực (realtime), do đó người dùng bắt buộc phải tải lại trang để cập nhật các phản hồi mới nhất.
- Chức năng tiện ích kéo-thả thư mục (drag-drop folder upload) hiện mới chỉ hoạt động tương thích trên các trình duyệt sử dụng nhân Chromium.
- Giao diện ứng dụng chưa thực sự được tối ưu hóa hiển thị (responsive) cho các thiết bị có kích thước màn hình nhỏ.
- Dự án chưa triển khai các quy trình kiểm thử phần mềm tự động như kiểm thử đơn vị (unit test) hay kiểm thử tích hợp (integration test).

3. Hướng phát triển

Dựa trên những hạn chế đã được nhận diện, nhóm đưa ra các định hướng phát triển và nâng cấp hệ thống trong tương lai, được chia thành ba nhóm chính:

- **Bổ sung tính năng mới:**
 - Tích hợp thư viện Socket.io để xử lý luồng bình luận theo thời gian thực (real-time comment).
 - Phát triển chức năng khôi phục mật khẩu thông qua cơ chế gửi email xác nhận (reset link).
 - Xây dựng hệ thống đánh giá chất lượng truyện theo thang điểm (rating từ 1 đến 5 sao).
 - Nghiên cứu thuật toán đề xuất truyện tự động dựa trên phân tích hành vi và lịch sử đọc của người dùng.
 - Đa dạng hóa chế độ đọc truyện (reading mode) như cuộn ngang (horizontal scroll) hoặc chế độ hiển thị trang đơn (single page).
- **Cải thiện và nâng cấp kỹ thuật:**
 - Áp dụng thuật toán mã hóa bcrypt cho toàn bộ mật khẩu người dùng để đảm bảo an ninh dữ liệu khi đưa lên môi trường production.
 - Thiết lập và thực thi các bộ kiểm thử tự động sử dụng framework Jest để thực hiện unit test và integration test.

- Triển khai các biện pháp bảo vệ máy chủ như giới hạn tần suất yêu cầu (rate limiting) và phòng chống tấn công từ chối dịch vụ (DDOS protection).
- Tích hợp hệ thống Redis để lưu trữ bộ nhớ đệm (caching), qua đó tăng tốc độ truy xuất dữ liệu hệ thống.
- Thiết lập quy trình tích hợp và triển khai liên tục (CI/CD pipeline) để tự động hóa vòng đời phát hành phần mềm.
- **Kế hoạch triển khai (Deployment):**
 - Đưa hệ thống Backend lên các nền tảng điện toán đám mây như Render hoặc Railway.
 - Triển khai giao diện Frontend thông qua các dịch vụ lưu trữ tối ưu như Vercel hoặc Netlify.
 - Thiết lập tên miền độc lập (custom domain) và cấu hình chứng chỉ bảo mật (SSL certificate) để mã hóa đường truyền.

TÀI LIỆU THAM KHẢO

- [1] “MERN Stack Explained,” MongoDB. Accessed: Feb. 27, 2026. [Online]. Available: <https://www.mongodb.com/resources/languages/mern-stack>
- [2] “Client Server là gì? Tìm hiểu mô hình Client Server từ A - Z.” Accessed: Jun. 15, 2026. [Online]. Available: <https://fptcloud.com/client-server/>
- [3] “What is RESTful API? - RESTful API Explained - AWS,” Amazon Web Services, Inc. Accessed: Mar. 07, 2026. [Online]. Available: <https://aws.amazon.com/what-is/restful-api/>
- [4] “Single Page Applications (SPA) điều hướng thế nào ?” Accessed: Jun. 15, 2026. [Online]. Available: <https://viblo.asia/p/single-page-applications-spa-dieu-huong-the-nao-4P856OL3KY3>
- [5] “MongoDB Courses and Trainings | MongoDB University.” Accessed: Mar. 07, 2026. [Online]. Available: <https://learn.mongodb.com/learn/course/modeling-data-relationships/conclusion/learn?page=2>
- [6] “Mongoose v9.3.1: Getting Started.” Accessed: Mar. 20, 2026. [Online]. Available: <https://mongoosejs.com/docs/>
- [7] “Express - Node.js web application framework.” Accessed: Feb. 27, 2026. [Online]. Available: <https://expressjs.com/>
- [8] Auth0, “JSON Web Tokens - jwt.io,” JSON Web Tokens - jwt.io. Accessed: Mar. 12, 2026. [Online]. Available: <https://jwt.io>
- [9] “Cloudinary Image & Video API Platform - Docs Home Page | Documentation.” Accessed: Feb. 27, 2026. [Online]. Available: <https://cloudinary.com/documentation>
- [10] *expressjs/multer*. (Mar. 20, 2026). JavaScript. expressjs. Accessed: Mar. 20, 2026. [Online]. Available: <https://github.com/expressjs/multer>
- [11] “axios | Promise based HTTP client.” Accessed: Jun. 15, 2026. [Online]. Available: <https://axios.rest/>
- [12] simo, *simov/slugify*. (Jun. 10, 2026). JavaScript. Accessed: Jun. 15, 2026. [Online]. Available: <https://github.com/simov/slugify>
- [13] “cors middleware · Express.js,” Express.js. Accessed: Jun. 15, 2026. [Online]. Available: <https://expressjs.com/en/resources/middleware/cors/>
- [14] “Quick Start – React.” Accessed: Feb. 27, 2026. [Online]. Available: <https://react.dev/learn>
- [15] “Vite,” vitejs. Accessed: Jun. 15, 2026. [Online]. Available: <https://vite.dev>
- [16] “Installing with Vite - Installation.” Accessed: Jun. 15, 2026. [Online]. Available: <https://tailwindcss.com/docs/installation/using-vite>
- [17] “Introduction - Zustand.” Accessed: Jun. 15, 2026. [Online]. Available: <https://zustand.docs.pmnd.rs/learn/getting-started/introduction>
- [18] “React Router Official Documentation.” Accessed: Jun. 15, 2026. [Online]. Available: <https://reactrouter.com/>
- [19] “react-hot-toast - The Best React Notifications in Town.” Accessed: Jun. 15,

2026. [Online]. Available: <https://react-hot-toast.com/>

[20] “React Icons.” Accessed: Jun. 15, 2026. [Online]. Available: <https://react-icons.github.io/react-icons/>

[21] “react-dropzone.” Accessed: Jun. 15, 2026. [Online]. Available: <https://react-dropzone.js.org/>